

学 号： 2022217586

密 级： 公开

合肥工业大学

Hefei University of Technology

本科毕业设计（论文）

UNDERGRADUATE THESIS



类 型：	论文
题 目：	遥感场景下的微小船舶检测系统设计 与实现
专业名称：	物联网工程
入校年份：	2022 级
学生姓名：	邝黄硕
指导教师：	惠天瑞 副教授
学院名称：	计算机与信息学院
完成时间：	2026 年 5 月

合 肥 工 业 大 学

本科毕业设计（论文）

遥感场景下的微小船舶检测系统设计与实现

学生姓名： 邝黄硕

学生学号： 2022217586

指导教师： 惠天瑞 副教授

专业名称： 物联网工程

学院名称： 计算机与信息学院

2026 年 5 月

A Dissertation Submitted for the Degree of Bachelor

**Design and Implementation of a Tiny Ship Detection
System in Remote Sensing Scenes**

By

Huangshuo Kuang

Hefei University of Technology

Hefei, Anhui, P.R.China

May, 2026

毕业设计（论文）独创性声明

本人郑重声明：所提交的毕业设计（论文）是本人在指导教师指导下进行独立研究工作所取得的成果。据我所知，除了文中特别加以标注和致谢的内容外，设计（论文）中不包含其他人已经发表或撰写过的研究成果，也不包含为获得 合肥工业大学 或其他教育机构的学位或证书而使用过的材料。对本文成果做出贡献的个人和集体，本人已在设计（论文）中作了明确的说明，并表示谢意。

毕业设计（论文）中表达的观点纯属作者本人观点，与合肥工业大学无关。

毕业设计（论文）作者签名：

签名日期：2026 年 5 月 30 日

毕业设计（论文）版权使用授权书

本学位论文作者完全了解 合肥工业大学 有关保留、使用毕业设计（论文）的规定，即：除保密期内的涉密设计（论文）外，学校有权保存并向国家有关部门或机构送交设计（论文）的复印件和电子光盘，允许设计（论文）被查阅或借阅。本人授权 合肥工业大学 可以将本毕业设计（论文）的全部或部分内容编入有关数据库，允许采用影印、缩印或扫描等复制手段保存、汇编毕业设计（论文）。

（保密的毕业设计（论文）在解密后适用本授权书）

学位论文作者签名：

指导教师签名：

签名日期：2026 年 5 月 30 日

签名日期：2026 年 5 月 30 日

摘要

遥感图像中的微小船舶检测常服务于海上监管、港口监测、海事执法和应急救援等任务。与普通自然图像不同，中分辨率遥感图像里船舶目标占像素少、纹理弱，附近又可能出现海浪、云层、岸线和港区设施等干扰，模型容易把背景当作船，也容易漏掉弱小目标。围绕这一类场景，本文以 LEVIR-Ship 数据集为实验对象，对不同检测路线进行比较，并实现一套遥感微小船舶检测系统。

算法部分先整理通用目标检测、小目标检测和遥感船舶检测的相关研究，再固定数据划分、评价指标和结果记录方式，选取 DRENet、FCOS 与 YOLO26 进行实验。本文以 AP50 作为主指标，同时记录 AP50-95、Precision、Recall 和 F1。测试结果中，DRENet 的 AP50 为 0.7949、Recall 为 0.8511，召回倾向明显但误检压力较大；YOLO26 的 AP50 为 0.7950、Precision 为 0.8430、AP50-95 为 0.3170，精度与定位质量更均衡；FCOS 的 AP50 为 0.7389、F1 为 0.7944，Precision 与 Recall 折中较好但整体检出能力弱于前两者。消融实验进一步分析了阈值和输入尺寸对模型输出的影响。

系统部分将推理功能封装为遥感微小船舶检测系统。系统前端采用 React，负责图像上传、任务查询和结果展示；后端采用 FastAPI，负责推理服务和任务接口；SQLite 保存模型信息、任务状态、检测框结果和输出文件路径。系统支持单图与批量任务、模型启停、异步执行、可视化结果回看以及历史任务查询，可以把离线实验中的检测结果放到可操作的页面流程中复查。

通过上述实验和系统实现，本文得到的判断是：在遥感微小船舶任务中，单看一个精度数值不足以解释模型行为，仍需要结合阈值设置、误检样例和任务结果一起分析。本文保留下来的实验记录、分析过程和系统原型，可作为后续模型优化、融合评测与工程整理的基础。

关键词：遥感图像；微小船舶检测；LEVIR-Ship；目标检测；检测系统

ABSTRACT

Tiny ship detection in remote sensing imagery is a basic component of maritime surveillance, port monitoring, law enforcement and emergency response. In mid-resolution images, ships often occupy only a few pixels and have weak texture cues, while waves, clouds, coastlines and harbor facilities introduce strong background interference. These factors make missed detections, false alarms and unstable cross-scene performance frequent in practice. Based on this setting, this thesis conducts comparative experiments on the public LEVIR-Ship dataset and implements a remote-sensing tiny ship detection system.

For the algorithmic study, this thesis reviews related work on general object detection, small object detection and remote-sensing ship detection, and then builds a unified protocol for data split, evaluation metrics and result recording. Under this protocol, DRENet, FCOS and YOLO26 are selected for comparison. AP50 is used as the primary metric, with AP50-95, Precision, Recall and F1 used as complementary indicators. The results show that DRENet achieves an AP50 of 0.7949 and a Recall of 0.8511, with a clear recall-oriented tendency but higher false-alarm pressure. YOLO26 achieves an AP50 of 0.7950, a Precision of 0.8430 and an AP50-95 of 0.3170, giving a more balanced result in precision and localization quality. FCOS achieves an AP50 of 0.7389 and an F1 of 0.7944, with a better Precision–Recall trade-off but lower overall detection capability than the other two. Ablation experiments further analyze the influence of threshold and input size.

For the system part, the inference capability is organized into a remote-sensing tiny ship detection system. The frontend is built with React for task submission and result visualization, the backend uses FastAPI for inference services and task management, and SQLite stores model information, task status, detection results and artifact paths. The system supports single-image and batch-image submission, model activation management, asynchronous task execution, detection-box visualization, structured result review and history query, so that offline inference outputs can be inspected through an interactive workflow.

The study suggests that unified multi-model evaluation and threshold ablation together provide a clearer view of the applicable boundaries of different detection paradigms in

remote-sensing tiny ship detection. The experimental records, analysis process and system prototype in this thesis can support later work on model optimization, fusion evaluation and engineering deployment.

KEYWORDS: remote sensing images, tiny ship detection, LEVIR-Ship, object detection, detection system

目 录

1 绪论	1
1.1 研究背景	1
1.2 研究问题与挑战	1
1.3 研究目标与主要内容	2
1.4 本文主要工作与贡献	3
1.5 技术路线与研究方法	3
1.6 论文结构安排	4
2 国内外研究现状	5
2.1 通用目标检测方法演进	5
2.2 小目标检测研究进展	5
2.3 遥感船舶检测研究现状	6
2.4 现有工作的不足与本文切入点	7
3 数据集、评价指标与方法方案	8
3.1 实验基础	8
3.2 数据集介绍	8
3.3 数据集样例与任务特征	8
3.4 数据预处理与统一格式	9
3.5 评价指标	10
3.6 指标选择依据	10
3.7 模型方案	11
3.7.1 DRENet (专项方法)	11
3.7.2 FCOS (Anchor-Free 检测路线)	12
3.7.3 YOLO26 (One-Stage 代表)	14
3.8 统一实验协议与实现约束	15

4 实验结果与分析	17
4.1 实验环境与设置	17
4.2 三模型主对比结果	17
4.3 效率与部署属性分析	19
4.4 模型逐项分析	20
4.4.1 DRENet 结果分析	20
4.4.2 FCOS 结果分析	20
4.4.3 YOLO26 结果分析	21
4.5 定性样例与误差分析	21
4.5.1 成功检测样例	21
4.5.2 漏检样例	22
4.5.3 误检样例	23
4.6 消融实验	24
4.6.1 阈值消融分析	24
4.6.2 输入尺寸敏感性分析	26
5 系统需求分析、设计与实现	27
5.1 系统建设目标	27
5.2 系统需求分析	27
5.2.1 功能需求	27
5.2.2 非功能需求	28
5.3 系统总体架构	28
5.4 数据库设计	30
5.5 接口设计	30
5.6 关键实现流程	31
5.6.1 任务创建与执行流程	31
5.6.2 结果生成与展示流程	31
5.7 模块实现细节	32

5.7.1 端到端数据流与职责分解	32
5.7.2 单图检测逻辑（接口层）	33
5.7.3 推理调用逻辑（服务层）	33
5.7.4 结果聚合逻辑（存储层）	34
5.8 系统实现效果.....	35
5.8.1 前端页面实现效果	35
5.9 系统输出案例集	36
5.10 模块测试用例设计	38
5.10.1 测试执行结果	38
5.11 系统响应速度优化.....	39
5.11.1 性能瓶颈分析与优化思路	39
5.11.2 并行推理与模型预加载机制	40
5.11.3 进度端点与前端轮询重构	40
5.11.4 渲染、存储与结果访问优化	41
5.11.5 优化效果分析	41
6 总结与展望	42
6.1 工作总结	42
6.2 不足分析	42
6.3 未来展望	43
6.4 全文结论	43
参考文献	44
致谢	46

插图清单

图 1.1 技术路线图	3
图 2.1 本文相关研究方向与代表路线脉络	6
图 3.1 LEVIR-Ship 数据集典型样例	9
图 3.2 数据预处理与统一格式流程图	10
图 3.3 DRENet 检测流程示意	11
图 3.4 DRENet 网络结构图（基于文献 ^[10] 图示翻译整理）	12
图 3.5 FCOS 检测流程示意	13
图 3.6 FCOS 网络结构图（基于文献 ^[4] 图示翻译整理）	14
图 3.7 YOLO26 检测流程示意	14
图 3.8 YOLO26 网络结构图	15
图 4.1 三模型主对比结果的指标可视化.....	18
图 4.2 三模型效率与复杂度对比柱状图.....	19
图 4.3 三模型多维度综合雷达图.....	20
图 4.4 三模型成功检测样例对比.....	22
图 4.5 三模型漏检样例对比.....	23
图 4.6 三模型误检样例对比.....	24
图 4.7 三模型阈值消融趋势图	25
图 4.8 输入尺寸敏感性趋势图	26
图 5.1 系统总体架构图	29
图 5.2 系统数据库 E-R 图.....	30
图 5.3 系统关键流程图	32
图 5.4 系统主要页面实现效果	36
图 5.5 样本 A 的系统输出结果图：目标边界相对清晰场景下的检测结果示例	37
图 5.6 样本 B 的系统输出结果图：弱纹理与低对比度条件下的检测结果示例	37
图 5.7 样本 C 的系统输出结果图：复杂背景干扰条件下的检测结果示例....	38

表格清单

表 2.1	本文相关研究方向与代表路线	7
表 3.1	LEVIR-Ship 数据集划分	8
表 4.1	实验环境与关键配置.....	17
表 4.2	三模型在 LEVIR-Ship 数据集上的主对比结果	18
表 4.3	三模型效率与复杂度对比.....	19
表 4.4	三模型阈值消融结果.....	25
表 4.5	FCOS 与 YOLO26 输入尺寸敏感性分析结果	26
表 5.1	功能需求的操作—响应定义	28
表 5.2	系统核心接口设计	31
表 5.3	系统主要模块测试用例	39
表 5.4	系统响应速度优化的关键基准结果	41

1 绪论

1.1 研究背景

遥感图像中的船舶检测，是海上监管、港口调度、海事执法与应急救援等应用中的基础环节。和自然场景目标检测相比，遥感图像的视野更大、背景更杂、目标更稀疏，尺度变化也更明显。依赖人工判读时，效率、一致性和可追溯性都不理想。随着深度学习目标检测技术的发展，将其引入遥感船舶检测任务，并继续把结果组织成可复验、可回放的系统链路，已经有了比较明确的研究和应用价值。

从任务定义看，遥感图像中的船舶检测属于标准目标检测问题：输入为遥感图像（单张或批量），输出为每张图像中的船舶目标集合，输出的每个目标包含边界框坐标、置信度和类别标识，并且由于遥感图像中目标普遍偏小、背景干扰强、尺度变化明显，该任务对检测模型的要求不仅是整体精度达标，还需要在召回率与精确率之间取得合理平衡——既要尽量检出真实船舶目标，又要控制近岸纹理、云层和港区设施等背景带来的误检。

近年来，目标检测技术从 two-stage 走到 one-stage、anchor-free，再到 transformer 检测器，模型在精度、速度和表达能力上持续演进^[1-7]。与此同时，遥感小目标检测也一直在关注尺度变化、背景干扰和样本稀疏对模型稳定性的影响^[8,9]。不过，在中分辨率遥感图像中，微小船舶占据的像素本就有限，还容易和海浪、云层、岸线或近岸建筑纹理混在一起，通用检测器直接迁移时往往会出现漏检和误检。

公开数据集和开源工具链逐步完善后，问题也从“模型能否训练出来”转向“结果能否统一评测、稳定复现并进入可运行流程”。只给出单一模型指标已经不够，仍需要在同一协议下比较不同检测范式，并把算法能力接入系统。本文的工作就是在这个背景下展开的。

1.2 研究问题与挑战

本课题讨论的是“遥感场景下的微小船舶检测系统设计与实现”。实际要解决的问题并不是单纯把某个模型的 AP50 做高，而是在弱纹理目标、复杂背景和可运行系统之间找到一个可验证、可解释的权衡。结合 LEVIR-Ship 数据集的图像特征以及 DRENet 等已有工作^[10]，本文把问题拆成以下几个较具体的难点。

首先是尺度问题。微小船舶在整幅图中只占很少像素，特征图经过多次下采样

后，可用于定位的细节更少。其次，海浪、云层、岸线和港区设施会形成大量近似船体的小纹理，模型一旦把这些纹理当作候选目标，**Precision** 就会明显受影响。再次，**LEVIR-Ship** 中不同图像块的目标密度、朝向和局部对比度差异较大，训练时正负样本比例并不稳定。还有一个容易被忽略的问题：数据划分、阈值和评测脚本只要不一致，跨模型比较就会变得很难解释。

本文在上述情况下将工作组织为三条并行路线，以保证实验结果、误差分析与系统展示之间的逻辑一致性，具体如下：

1. **实验协议线**：固定数据划分、评价指标、阈值设置和结果字段，并对配置文件、权重版本和运行日志进行统一记录，用以降低实验口径差异带来的偏差，使模型结果具备可比性和可复验性。
2. **模型比较线**：在统一协议下选取 **DRENet**、**FCOS** 和 **YOLO26** 三类代表模型，针对 **AP50**、**Precision**、**Recall**、**F1** 以及效率指标进行横向比较，并结合成功样例、漏检样例和误检样例分析性能差异来源。
3. **系统落地线**：将离线推理能力接入任务管理与可视化流程，形成从任务提交、异步执行到结果回看的完整链路，以验证模型结果在系统环境中的可用性与可追溯性，并为后续应用场景提供工程化支撑。

1.3 研究目标与主要内容

本文的目标可以概括为：围绕特定的 **LEVIR-Ship** 数据集整理一套可复查的数据和评测结果，首先完成 **DRENet**、**FCOS** 与 **YOLO26** 的对比实验，然后构建相应的 **Web** 系统来用于模型推理、结果展示与历史记录管理。

具体工作包括：

1. **整理数据与评测口径**：固定数据划分、指标计算方式和结果记录字段，减少模型比较中的人为差异；
2. **完成三类模型实验**：基于 **DRENet**、**FCOS** 与 **YOLO26** 的训练记录、测试和正式结果来比较不同检测路线的表现；
3. **补充误差与消融观察**：结合阈值、输入尺寸和可视化样例来分析误检、漏检和原因；
4. **搭建检测系统原型**：使用 **React**、**FastAPI** 与 **SQLite** 来构建 **Web** 系统以实现任务提交、异步执行、结果保存和历史回看。

1.4 本文主要工作与贡献

根据当前完成的实验和系统，本文的工作重点主要体现在以下三方面。

第一，围绕遥感微小船舶检测任务，本文整理了数据组织和评测记录方式。数据划分、输入口径、评价指标和结果字段被放到同一套约束下，后续表格中的模型差异才有相对稳定的比较基础。

第二，本文完成了 DRENet、FCOS 与 YOLO26 的对比实验，并结合阈值消融与定性样例分析模型在不同场景下的表现。这样处理后，AP50 接近但 Precision、Recall 差别很大的情况就不再只是表格中的数字，而可以对应到具体错误类型和部署取舍。

第三，本文将离线推理能力整合到前后端分离的 Web 系统中。任务提交、异步执行、检测框可视化、结果持久化和历史回看都已完成，关键检测结果可以通过系统页面完成查询和回看。

1.5 技术路线与研究方法

图 1.1 给出了本文的技术路线，实际进展顺序是首先核对数据与标注，再完成模型训练和推理并整理实验结果，最后系统接入推理接口，本文通过统一的数据记录和推理入口保证算法实验与系统实现，能够让定性样例、结构化结果和历史回看保持一致。

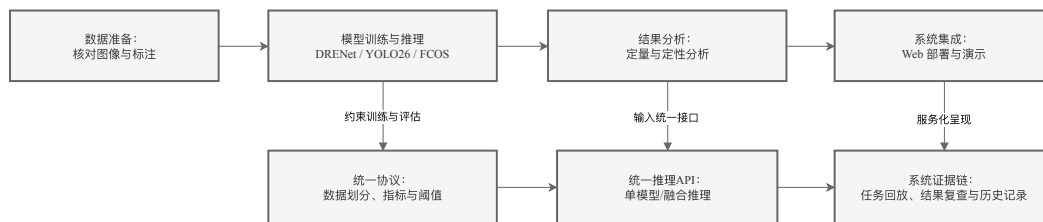


图 1.1 技术路线图

落到具体操作上：在数据侧先核对图像和标注是否一一对应，再整理成后续框架都能用的中间格式；在模型层面选取专项方法、anchor-free 和 one-stage 三条路线进行同口径比较；分析时不只以 AP50 为依据，也把误检样例、漏检样例和消融结果结合起来观察模型行为；在系统侧则考察推理能力能否在任务提交、状态查询和结果展示的实际流程中稳定运行。

本文沿着这条路线最终形成了三类产物：LEVIR-Ship 上的数据与评测记录，

DRENet/FCOS/YOLO26 的对比结果和误差分析以及一个可运行的 Web 检测系统，后文各章也将围绕这三类产物展开。

1.6 论文结构安排

全文共分为六章，各章内容安排如下。

1. 第一章为绪论，介绍研究背景、问题挑战、研究目标、主要工作以及技术路线；
2. 第二章为国内外研究现状，梳理通用目标检测、小目标检测与遥感船舶检测相关研究，并明确本文切入点；
3. 第三章介绍数据集、评价指标与方法方案，说明统一实验协议、模型选择依据以及实现约束；
4. 第四章给出实验结果与分析，展示三模型主对比结果、效率对比、定性样例与消融实验；
5. 第五章介绍系统需求分析、总体设计与实现过程，说明系统架构、数据库、接口与关键流程；
6. 第六章为总结与展望，归纳全文工作，分析当前边界，并给出后续研究方向。

2 国内外研究现状

2.1 通用目标检测方法演进

通用目标检测的发展可以先从 R-CNN 系列看起。Girshick 等人的工作^[11]把候选区域、特征提取和分类回归串成一条检测流程，后来 Faster R-CNN^[1]又把候选框生成进一步网络化，成为 two-stage 路线中很常被引用的代表。这一路线通常定位较稳，但候选区域处理会带来额外计算，推理速度不如更直接的单阶段方法。

one-stage 方法与之相比取消了显式候选区域生成过程且直接在特征图上进行密集预测，YOLO 系列^[5, 12]、RetinaNet^[3]和 EfficientDet^[13]都属于该路线的代表工作，它们结构紧凑、推理速度快、部署方便，因此在工程场景中应用较多；但当遇到复杂背景和弱目标时，误检与漏检问题仍需要单独分析。

anchor-free 方法的出发点是减少 anchor 尺寸、比例和匹配策略带来的调参负担。FCOS^[4]直接在特征图位置上回归目标框，并用 centerness 抑制偏离目标中心的预测。这个设计在通用场景中较有代表性，但遥感微小船舶的目标面积太小，anchor-free 方法能否稳定优于 anchor-based 或专项方法，还需要放到同一任务口径下测试。

另一条近年的路线是 transformer 检测器。DETR^[6]将目标检测组织成端到端集合预测问题，Deformable DETR^[7]和 DINO^[14]继续改进收敛速度和检测精度。这类方法的全局建模能力较强，不过训练资源、数据规模和调参时间要求也更高。考虑到本课题需要同时完成实验比较和系统实现，本文将作为方法脉络的一部分讨论，没有把它列为主要复现对象。

把这些方法放到一起看，差异不只体现在精度上，还体现在速度、输入设置和调参成本上。本文选择 DRENet、FCOS 与 YOLO26，就是为了在专项方法、anchor-free 方法和 one-stage 方法三条路线之间进行同口径比较。

2.2 小目标检测研究进展

小目标检测困难的原因，首先是因为可用像素少导致目标在特征图上经过下采样后可能只剩很小响应，纹理、边缘和上下文都不稳定；同时图像中大量背景区域会放大正负样本不平衡，围绕这些问题，相关研究常从下述几类思路展开。

一类做法直接改进特征表达。FPN^[2]将浅层细节和深层语义结合起来，后来许多小目标检测方法都沿用了这种多尺度思想。遥感场景中，SCRDet^[15]等方法进一步

结合注意力和旋转检测，以适应小尺度、密集分布目标。这类方法容易和现有检测器组合，但遇到极弱纹理目标时，特征仍可能被背景响应盖住。

另一类思路从尺度入手。SNIP^[16]、SNIPER^[17] 和 AutoFocus^[18] 的结果说明，训练样本选取、多尺度裁块和推理调度都会影响小目标表现。这类策略往往能改善候选区域分配，但实验流程随之变复杂。如果不同模型使用不同尺度策略，后续横向比较就必须格外谨慎。

还有一些工作不大改网络，而是调整训练目标和样本分布。Focal Loss^[3] 降低易分类样本权重，让模型更多关注困难样本；数据增强、困难样本重采样和损失重加权也常被用于小目标训练。它们实现成本相对低，但对数据质量和任务分布比较敏感。

最后一类是后处理和推理策略。小目标稀疏、背景又复杂时，置信度阈值、NMS 和融合策略会直接改变 Precision 与 Recall 的平衡。本文后续设置阈值消融，也是因为这些参数并非单纯的工程附属项。

已有综述^[8, 19] 也指出，遥感小目标检测还受到旋转变化的影响、密集分布、复杂背景纹理和标注成本影响。因此，本文分析模型差异时不只讨论网络结构，也把输入尺度、样本分布和后处理参数一并纳入。

2.3 遥感船舶检测研究现状

船舶检测是遥感目标检测中应用较早、需求也比较明确的一类任务。随着公开数据集和开源框架增多，研究重心已经从手工特征逐步转到深度学习检测器。Zhao 等人^[9] 对光学遥感船舶检测做过系统梳理，其中一个结论是：这类任务既继承通用检测框架，又必须处理近岸干扰、尺度变化、旋转形态和复杂海况等遥感特有难题。本文相关研究方向与代表路线脉络如图 2.1 所示。

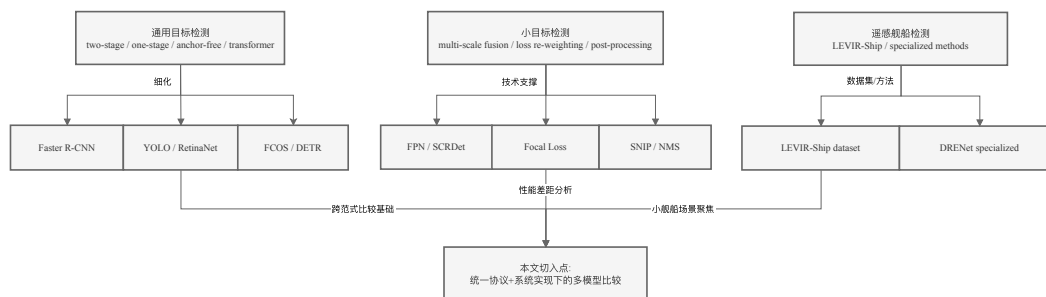


图 2.1 本文相关研究方向与代表路线脉络

LEVIR-Ship 数据集^[10] 聚焦中分辨率遥感图像中的微小船舶，为不同方法提供了同一数据基础。DRENet 也是围绕该数据集提出的专项方法，其退化重建增强机制主要服务于训练阶段，希望让网络对弱纹理船舶目标更敏感。本文选择它作为主线模型之一，正是因为它和本文场景最贴近。

工程工具链的成熟也改变了这类课题的组织方式，MMDetection^[20] 和 Ultralytics^[21] 提供了训练、测试和部署所需的基础设施，使得不同模型之间的复现和比较更方便，相关工作也慢慢开始重视评测协议、复现记录和实验流程的一致性，而这种一致性通常也需要借助 COCO、VOC 等通用基准协议来统一理解^[22, 23]。

本文从通用目标检测、小目标检测和遥感船舶检测三条路线展开，各个路线的代表方法与本文之间的关系见表 2.1。

表 2.1 本文相关研究方向与代表路线

研究方向	代表路线	本文定位
通用检测	Faster R-CNN / FCOS / YOLO / DETR	作为跨范式对比基础
小目标检测	多尺度融合、损失重加权、增强策略	用于分析性能差异来源
遥感船舶检测	LEVIR-Ship + 专项方法（如 DRENet）	聚焦微小船舶场景
工程化实现	MMDetection / Ultralytics / 服务化封装	支撑系统实现

2.4 现有工作的不足与本文切入点

从上述研究看，遥感微小船舶检测已有不少方法积累，但落到本文这样的多模型比较和系统实现任务时，仍有几个问题需要单独处理。

第一，实验协议通常不完全一致。数据划分、输入尺度、阈值和评测脚本若稍有不同，结果就很难直接并排解释。第二，一些研究对训练配置和日志记录交代不够清楚，复现实验时容易卡在工程细节上。第三，误差分析经常停留在总体指标，复杂海况、近岸背景和弱纹理目标造成的误检漏检没有被充分展开。其四，离线实验结果进入任务管理、可视化和历史回看流程的路径，仍有不少工作可以进一步整理。

针对这些问题，本文以统一协议下的多模型比较为主线，选取了 DRENet、FCOS 和 YOLO26 作为三个参照模型，并且结合指标、样例和消融结果讨论它们的适用范围。最后构建相应的 Web 图像推理系统，使算法结果能够进入任务管理、可视化和历史回看流程。

3 数据集、评价指标与方法方案

3.1 实验基础

本章说明数据集来源、预处理方式、评价指标、模型选择和实验口径，为后续实验结果与系统实现提供统一基础。

3.2 数据集介绍

本文的实验统一采用 LEVIR-Ship 数据集^[10]，此数据集主要是面向中分辨率遥感微小船舶检测任务，在复杂海面背景下它能够反映小目标检出的实际难点，数据集划分见表 3.1。

表 3.1 LEVIR-Ship 数据集划分

数据子集	图像数量	目标数量
训练集	2321	2002
验证集	788	665
测试集	788	552

LEVIR-Ship 里船舶目标普遍偏小，背景纹理又比较杂。有些图像中，船体跟海浪、岸线或港区设施的纹理差异本身就不大，后处理阈值一调，模型输出跟着变。正因为这样，本文在实验设计里没有只拿一个主指标去判模型好坏，而是把检出能力、误检控制和工程部署相关指标都记下来。

3.3 数据集样例与任务特征

LEVIR-Ship 涵盖了开阔海域、近岸区域、港口区域和多目标密集分布等场景，不同图像块之间背景纹理、目标密度、朝向和局部对比度的差异都比较大。模型一方面要对微弱目标保持敏感，另一方面又得压住复杂背景里的伪目标响应。“目标小而背景强”这个矛盾，基本就是遥感微小船舶检测的核心难点。

不同场景的样例图像见图 3.1。后续实验不只看 AP50，也会记录 Precision、Recall 和 F1，同时保留成功检测、漏检和误检样例，方便观察模型在不同背景条件下的具体表现。



图 3.1 LEVIR-Ship 数据集典型样例

3.4 数据预处理与统一格式

在跨框架实验中，最容易出差错的地方往往是数据和评测脚本，而不是网络结构，所以为了减小这部分误差，实验前先统一数据预处理和结果组织方式。

处理流程大致为：先核对图像和标注文件是否一一对应，并检查空标注、异常坐标和类别不一致等情况；然后把数据同步整理成 COCO 中间格式，让 MMDetection 路线和 YOLO 路线尽量共用同一套评测基础；再固定 train、val、test 划分及样本清单，防止模型之间因为样本范围不同而产生额外差异；最后，各模型输出统一整理成包含 image_id、model_name、bbox、score 和 category_id 的结构化记录，供可视化和数据库存储使用。

上述处理的目的在于：尽量让差异来自检测范式和实现策略，而不是来自数据组织或脚本细节。若每个模型都使用不同划分、不同评测脚本和不同输出字段，后续得到的数值就很难解释。统一格式和统一口径在本文中属于实验方法的一部分，不只是训练前的文件整理。

统一格式也直接服务于系统实现。不同框架与实现方式的推理结果可以进入同一套后处理与可视化逻辑；结构化结果也能写入 Web 系统数据库，使离线实验结果和在线任务回放使用同一类数据表达。数据预处理与统一格式流程见图 3.2。

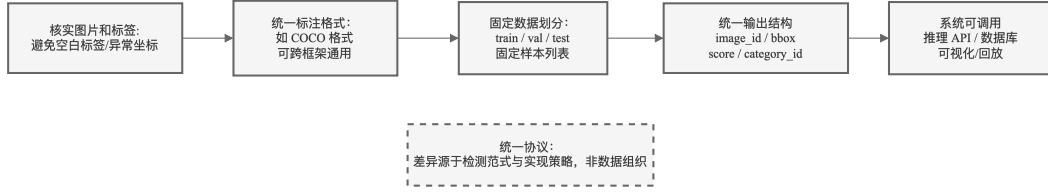


图 3.2 数据预处理与统一格式流程图

3.5 评价指标

本文实验使用 AP50 作为主指标，AP50-95、Precision、Recall 与 F1 作为辅助指标，他们的相关定义如下。

$$\text{Precision} = \frac{TP}{TP + FP} \quad (1)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2)$$

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (3)$$

$$\text{IoU} = \frac{|B_{pred} \cap B_{gt}|}{|B_{pred} \cup B_{gt}|} \quad (4)$$

其中，AP50 对应 IoU 阈值为 0.5 时的平均精度，本文用它观察模型是否能把微小船舶找出来；AP50-95 覆盖多个 IoU 阈值，对边框位置质量更敏感；Precision 更接近误检控制情况，Recall 更接近漏检控制情况，F1 用于观察二者折中。本文还记录 FPS、参数量和 FLOPs，用于讨论模型接入系统时的部署成本。相关评测口径与 IoU 阈值区间设置与 COCO 指标定义保持一致^[22]。

3.6 指标选择依据

AP50 放在主指标位置，主要是因为船舶目标太小。预测框和标注框只要偏移几个像素，IoU 就可能明显变化。若过度依赖高 IoU 阈值，边框回归误差会被放大，“模型是否发现目标”反而看不清楚。AP50 用来回答基本检出能力，AP50-95 则补充观察定位质量。

Precision、Recall 和 F1 在本文中同样重要。DRENet、FCOS 与 YOLO26 的设计出发点不同，只按 AP50 排序，容易掩盖误检和漏检之间的取舍。Recall 高的模型适合目标排查类场景，Precision 高的模型更适合输出需要稳定可控的系统展示场景。后续分析会把 Precision-Recall 平衡作为解释模型行为的主要依据之一。

3.7 模型方案

为了让对比不局限在同一类检测器内部，本文选取三类模型：DRENet 对应遥感微小船舶专项方法，FCOS 对应 anchor-free 检测路线，YOLO26 对应 one-stage 路线。

3.7.1 DRENet（专项方法）

DRENet 是针对 LEVIR-Ship 数据集提出的专项方法^[10]，通过在训练阶段引入退化重建增强机制，目的是让网络更关注弱纹理船舶目标，而本文将 DRENet 作为主模型之一，也会重点观察它在 Recall、复杂背景和召回优先任务中的表现。

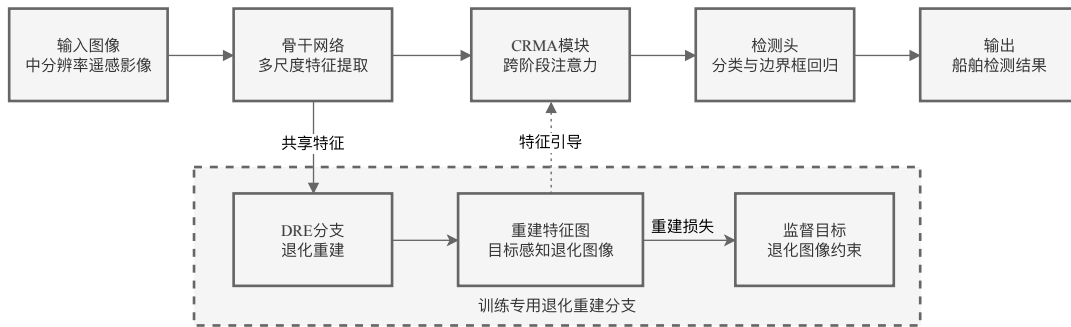


图 3.3 DRENet 检测流程示意

从神经网络工作机理看，DRENet 由“检测主干路径 + 训练期增强路径”两部分构成。检测主干负责常规定位与分类；增强路径通过退化-重建任务向骨干特征注入弱纹理目标的判别信息，并在训练阶段提供附加监督。该设计的直接效果是提升模型对微小、低对比船舶目标的敏感性。

DRENet 的突出创新点主要体现在以下方面：

1. **退化重建增强机制**：训练阶段对输入图像施加模拟退化（模糊、噪声、分辨率下降等），再要求网络从退化版本中重建原始外观，该辅助分支与检测主干共享同一骨干网络，使骨干同时接受检测损失与重建损失的双重监督，从而显式强化对弱纹理目标的判别能力——即使目标在退化后几乎不可辨认，网络仍被约

束去恢复其特征。

2. **跨阶段区域记忆增强模块（CRMA）**：该模块将浅层特征图（保留空间细节与边缘信息）与深层特征图（承载语义判别信息）进行跨阶段连接与融合，使小目标的定位信号不会在下采样过程中被完全淹没，相比仅依赖 FPN 的自顶向下路径，CRMA 提供了更直接的浅层—深层协同通道。
3. **双路径联合训练策略**：检测路径与重建路径在训练期并行运行，推理时只保留检测路径，不增加额外推理开销，这种”训练期增强、推理期无负担”的设计使 DRENet 在保持推理效率的同时获得了对弱纹理目标的更高敏感性。

对应到实验现象，DRENet 通常 Recall 更高，但在复杂背景下误检压力也更大，因此更适合作为召回优先路线。

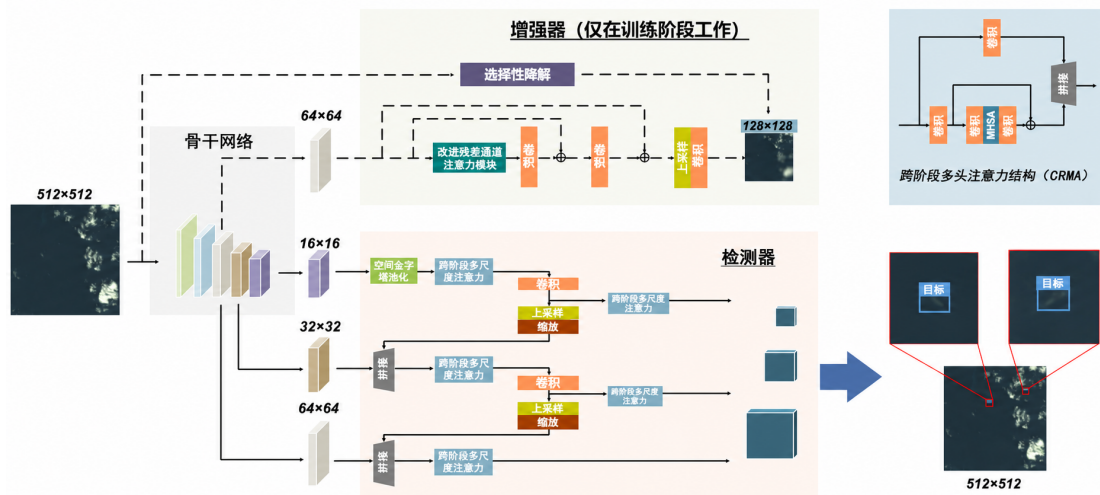


图 3.4 DRENet 网络结构图（基于文献^[10] 图示翻译整理）

3.7.2 FCOS（Anchor-Free 检测路线）

FCOS 用位置回归替代锚框设计，减少了 anchor 尺寸、比例和匹配策略等超参数依赖^[4]。本文将 FCOS 作为 anchor-free 路线的代表模型，与专项方法和 one-stage 路线放在同一评测口径下比较，观察其在遥感微小船舶任务上的实际表现。

FCOS 的基础机理是 anchor-free 密集回归。输入图像经 Backbone 与 FPN 生成多尺度特征后，检测头在每个位置直接预测边框偏移和类别分数，并通过 centerness 分支评估预测质量。最终由分类分数与 centerness 联合排序，再经 NMS 输出检测结果。

FCOS 的核心创新点主要体现在以下方面：

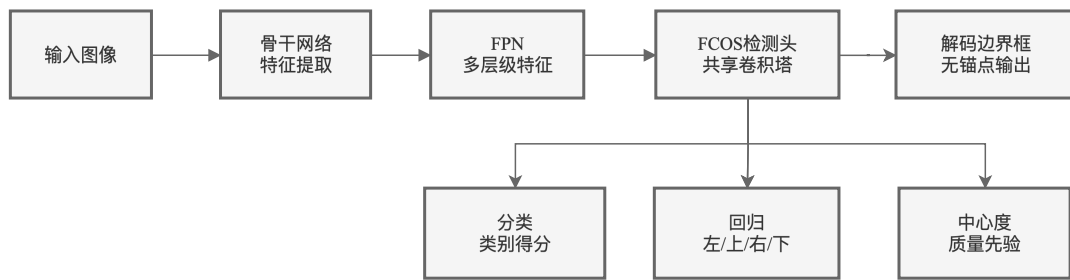


图 3.5 FCOS 检测流程示意

1. **逐点预测替代锚框匹配**：FCOS 在特征图每个位置直接预测该点到目标边界框四条边的距离偏移（上、下、左、右），以及类别概率，完全取消了锚框的尺度、比例和匹配阈值等超参数设计，这一改变降低了跨数据集迁移时的调参负担，也避免了锚框匹配中正负样本定义不一致的问题。
2. **centerness 分支抑制低质量候选**：远离目标中心的特征点往往产生定位不准确的预测框，传统 anchor-free 方法中这些低质量候选会拉低整体排序效果，FCOS 引入 centerness 分支，预测每个点距目标中心的相对距离，并将 centerness 与分类分数相乘作为最终排序依据，从而有效抑制边缘区域的低质量候选框，提升检测结果的整体排序稳定性。
3. **基于 FPN 层级的尺度分配策略**：FCOS 为不同 FPN 层级显式指定目标尺度范围，每个层级只负责特定大小区间的目标检测，减少了不同层级之间的预测重叠与竞争，使小目标在浅层层级获得更充分的响应空间。

对本课题而言，FCOS 的优势是训练口径清晰、Precision-Recall 折中较稳，不足是对输入尺寸较敏感——当推理输入过小时，特征图分辨率不足以支撑逐点回归的精度。

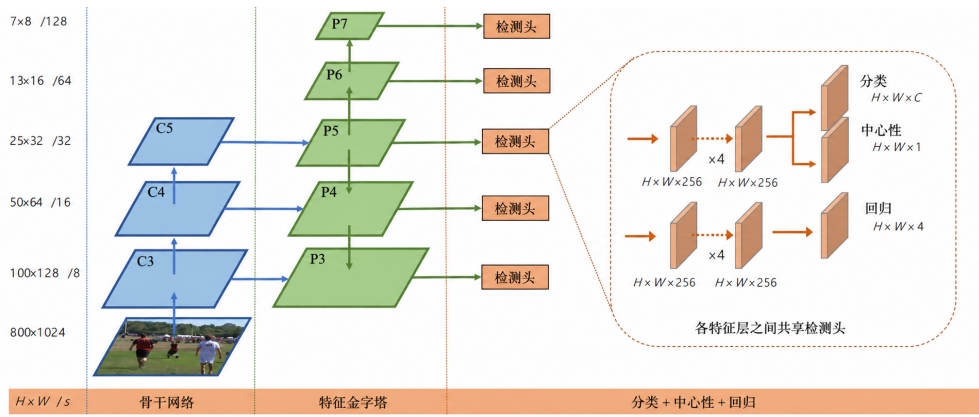


图 3.6 FCOS 网络结构图（基于文献^[4] 图示翻译整理）

3.7.3 YOLO26（One-Stage 代表）

YOLO 系列以单阶段预测和推理效率见长，在工程部署中使用较多^[5, 12, 21]。本文选取 YOLO26 代表 one-stage 路线，关注它在检测精度、边框质量、训练链路成熟度和系统接入成本之间的表现。

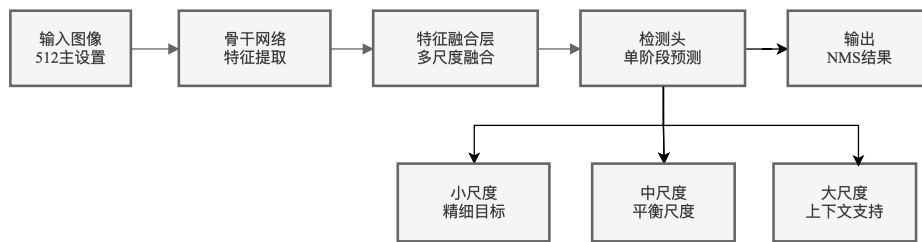


图 3.7 YOLO26 检测流程示意

YOLO26 延续 one-stage 端到端范式：Backbone 提取语义特征，Neck 执行跨尺度融合，检测头在多个尺度同时输出候选框与类别分数，单次前向即可完成检出。该结构在工程上具备较好的吞吐效率与部署友好性。

YOLO 路线的突出创新点主要体现在以下方面：

1. **多尺度检测头增强小目标可见性**：YOLO26 在三个不同尺度的特征图（P3、P4、P5）上同时设置检测头，浅层特征图分辨率较高，能够保留更多小目标的细节信息，使微小船舶在浅层层级获得更充分的候选框生成机会，相比只在单一尺度输出的检测器，多尺度检测头对小目标的覆盖更完整。
2. **结构轻量化与高效模块设计**：YOLO26 采用 C2f 等高效构建模块替代传统重参

数卷积, 在保持特征表达能力的同时控制参数量与计算量, 本文实验中 YOLO26 的参数量仅为 2.5M、FLOPs 为 3.7G, 远低于 FCOS 的 32.2M 与 123.0G, 使它在部署场景中具备更高的吞吐效率。

3. 任务对齐分配与训练增强策略: YOLO26 采用任务对齐分配 (TaskAlignedAssigner) 替代传统 IoU 匹配策略, 将分类质量与定位质量联合评估来分配正负样本, 使训练阶段的样本定义更贴合最终评测目标。同时配合 mosaic、mixup 等数据增强策略, 提升模型对稀疏小目标和复杂背景的泛化能力。

对本文任务而言, YOLO26 的优势是输出较干净、部署链路成熟, 适合作为系统默认候选; 它与 DRENet 在误检控制上也能形成互补。

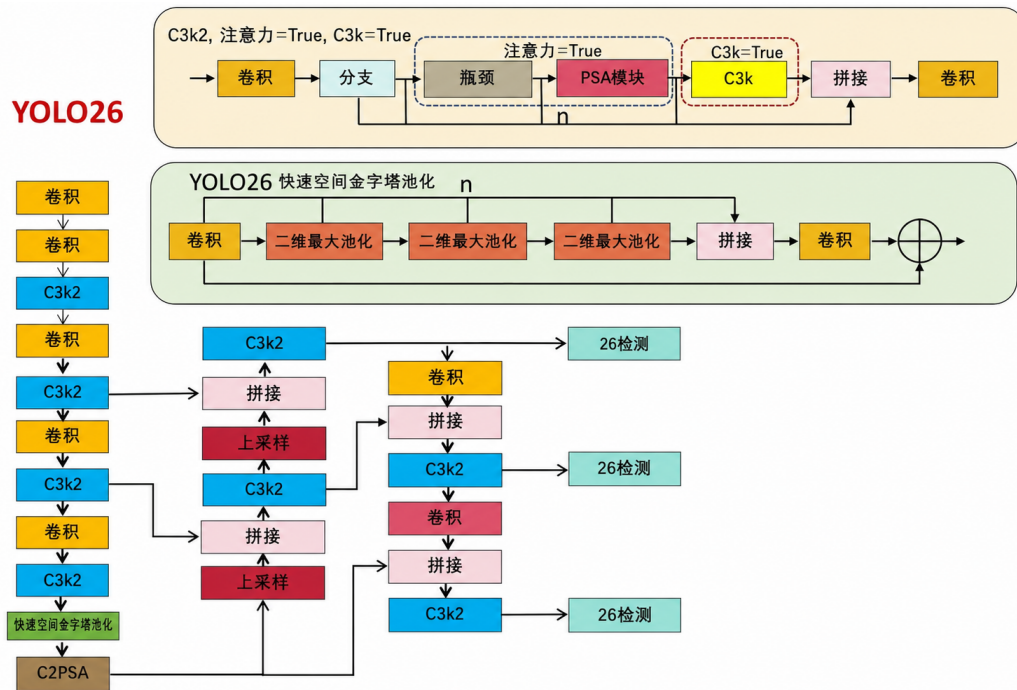


图 3.8 YOLO26 网络结构图

3.8 统一实验协议与实现约束

为了让后续比较有明确边界, 本文对实验协议作了几项约束: 数据划分和评测口径保持一致; 实验环境、配置文件、权重路径、日志文件和关键结果都保留记录; 结果输出结构也统一整理, 便于系统侧调用。

需要说明的是, 统一协议不是把所有模型改造成完全相同的实现, 本文是将统一的评测流程来应用到不同模型上, 以便比较不同模型在同一任务中的表现。

在实现层面，本文通过统一推理入口封装不同模型能力：

```
predict(image_path, model_name, conf_threshold, iou_threshold)
```

在此基础上，本文继续封装融合推理接口和系统任务执行逻辑。模型训练后的离线评测、命令行单图推理、批量推理以及 Web 系统接入，都采用同一个核心入口。

4 实验结果与分析

4.1 实验环境与设置

本章使用基于 LEVIR-Ship 数据集的实验记录,对模型 DRENet、FCOS 和 YOLO26 进行比较,数据指标保留了 AP50、AP50-95、Precision、Recall 与 F1;工程方面记录了 FPS、参数量和 FLOPs,训练配置、权重文件、运行日志和结果文档也均已保存,后续表格和图示可以追溯到对应产物。

除离线训练和测试外,系统侧推理流程也完成联调,相关输出可用于第五章的任务管理与结果回看说明。

本文实验的硬件、软件和关键超参数记录如表 4.1 所示。

表 4.1 实验环境与关键配置

配置项	设置说明
操作系统	macOS (本地联调) + Linux (云端训练)
主要计算设备	本地 CPU/MPS (推理与验证); 云端 NVIDIA RTX 3080 Ti 12GB (主训练)
后端框架	PyTorch, MMDetection (FCOS), Ultralytics (YOLO26), DRENet 原始实现
系统框架	FastAPI + React + SQLite
数据集与划分	LEVIR-Ship, 固定 train/val/test 划分 (2321/788/788)
输入尺寸 (主口径)	DRENet: 512; YOLO26: 512; FCOS: 800
置信度与 IoU 阈值	默认 conf=0.25、iou=0.50 (另做阈值消融)
主要评测指标	AP50 (主指标)、AP50-95、Precision、Recall、F1
效率指标采集口径	FPS 统一在单图推理场景统计; 参数量与 FLOPs 按模型结构计算并统一换算单位
结果记录与追踪	保留配置文件、权重路径、运行日志、结构化预测结果和可视化产物

4.2 三模型主对比结果

三模型核心结果见表 4.2, 对应日志和权重均已留存。

表 4.2 三模型在 LEVIR-Ship 数据集上的主对比结果

模型	AP50	AP50-95	Precision	Recall	F1
DRENet	0.7949	0.2919	0.4927	0.8511	0.6241
FCOS	0.7389	0.2880	0.7621	0.8297	0.7944
YOLO26	0.7950	0.3170	0.8430	0.7220	0.7778

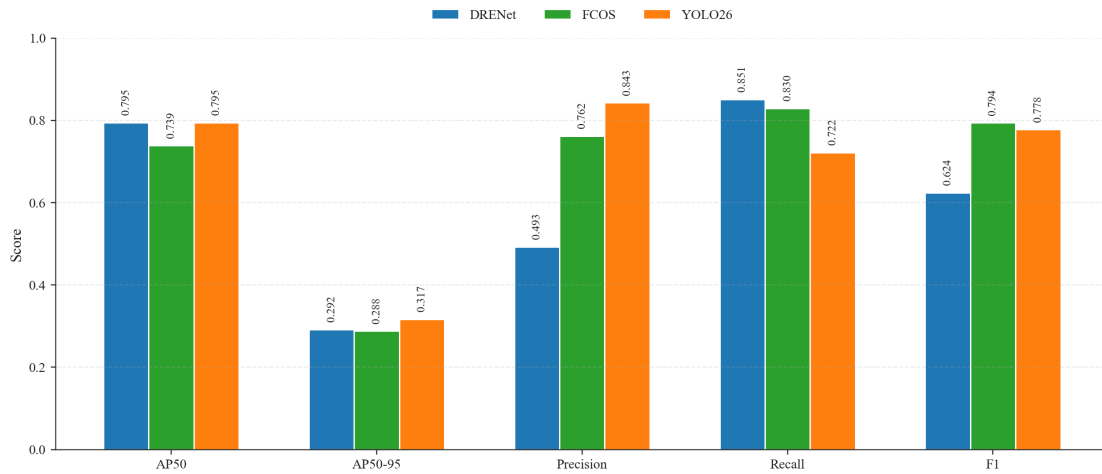


图 4.1 三模型主对比结果的指标可视化

表 4.2 与图 4.1 中，YOLO26 和 DRENet 的 AP50 几乎相同，分别为 0.7950 和 0.7949，而 FCOS 为 0.7389，故仅从 AP50 看，专项方法和成熟的 one-stage 路线在该数据集上都能完成较稳定检出，而 FCOS 与前两者依然有明显差距。

差异主要出现在 AP50-95、Precision 和 Recall 上。YOLO26 的 AP50-95 为 0.3170，高于 DRENet 的 0.2919 和 FCOS 的 0.2880，说明在更严格的 IoU 阈值下，它的边框位置更稳。DRENet 的 Recall 为 0.8511，高于 YOLO26 的 0.7220 和 FCOS 的 0.8297，但 Precision 只有 0.4927，低于 YOLO26 的 0.8430 和 FCOS 的 0.7621。也就是说，DRENet 更愿意保留可疑目标，代价是误检更多；YOLO26 对候选框筛选更严格，输出更干净；FCOS 的 Precision 和 Recall 折中较好，F1 为 0.7944，但整体检出能力弱于前两者。

4.3 效率与部署属性分析

在工程部署场景下，除精度指标外，还要关注模型在速度和复杂度方面的表现，当前可用效率结果如表 4.3 所示。

表 4.3 三模型效率与复杂度对比

模型	FPS	Params(M)	FLOPs(G)
DRENet	121.8993	4.7882	4.2057
FCOS	45.9559	32.1664	123.0
YOLO26	69.0625	2.5042	3.6939

对于三个模型在效率表中的趋势和精度表并不完全相同，DRENet 在实验记录中 FPS 最高，说明其推理速度最快，YOLO26 的参数量为 2.5042M、FLOPs 为 3.6939G，模型的规模最小，而 FCOS 的参数量和计算量较高，故整体效率低于前两者，三模型效率与复杂度对比的可视化见图 4.2。

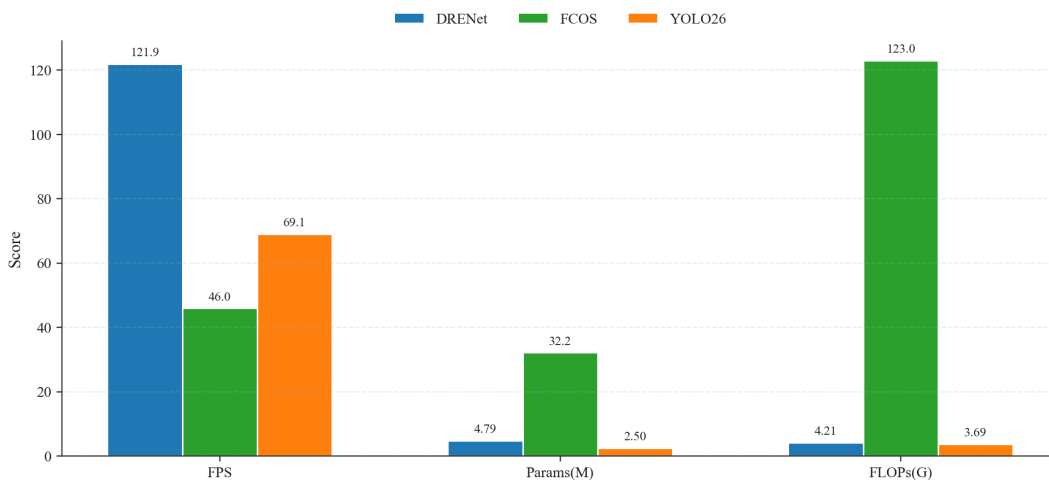


图 4.2 三模型效率与复杂度对比柱状图

把精度和效率放在一起读，DRENet 在召回和速度方面优势明显，YOLO26 在定位质量、误检控制和模型规模之间更均衡，FCOS 的 Precision-Recall 折中较好但整体检出能力和效率偏低。为综合展示三模型在精度与效率各维度上的表现，本文绘制了多维度雷达图（见图 4.3），其中参数效率和计算效率取逆指标（越小越优）进行归一化。

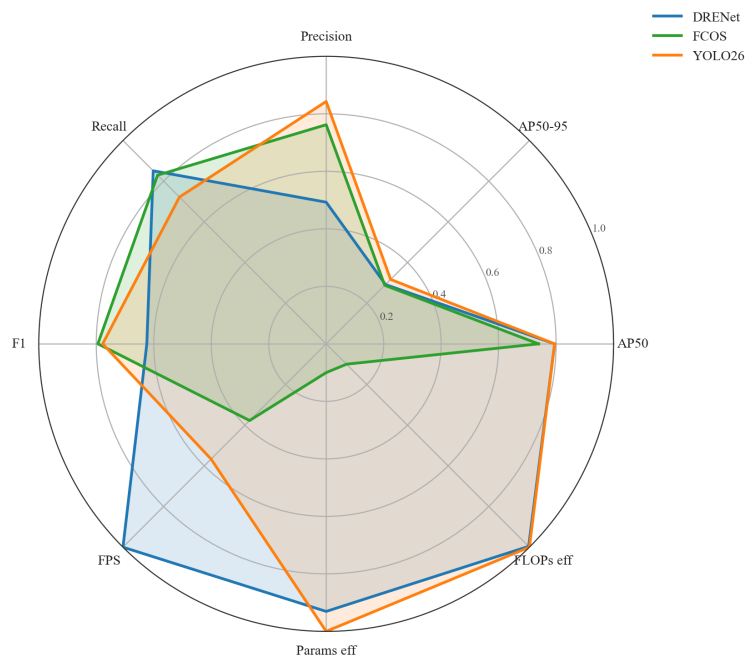


图 4.3 三模型多维度综合雷达图

4.4 模型逐项分析

4.4.1 DRENet 结果分析

DRENet 模型在上述结果中的特点是 Recall 高, 其 Recall 为 0.8511, 高于 YOLO26 模型, 说明该模型对于复杂背景和弱纹理目标更倾向于保留候选框, 所以如果任务目标是先尽可能发现可疑船舶再由后续流程复核, 这种结果取向更有实际价值。

而 DRENet 的问题为, Precision 较低, 数值为 0.4927, 误检的可能性较大, 对于近岸区域、港区设施和背景边缘纹理都可能被它保留下来, 所以 DRENet 更适合作为召回优先的模型候选方案。

4.4.2 FCOS 结果分析

FCOS 模型的 AP50 结果为 0.7389, 数值低于 DRENet 与 YOLO26; 但它的 Precision 为 0.7621、Recall 为 0.8297、F1 为 0.7944, 表明了在当前评测配置下, FCOS 能更好的保留正样本, 其 Precision 与 Recall 的折中表现优于 DRENet, 但是 FCOS 的参数量 (32.17M) 和计算量 (123.0G) 均明显高于另外两个模型, 同时他的推理速度也最慢 (45.96 FPS), 所以整体推理效率偏低, 在遥感微小船舶任务中, FCOS 的

检出能力比不上 DRENet 和 YOLO26，但是它的 Precision-Recall 平衡和 F1 稳定性仍对本文具有一定的参考价值。

4.4.3 YOLO26 结果分析

YOLO26 的 AP50 与 DRENet 几乎相同，但它的 AP50-95、Precision 和 F1 更高，其中 Precision 达到 0.8430，说明在当前阈值设置下，它能够一定程度上避免检出复杂背景中的伪目标。

YOLO26 也更偏向工程侧，其训练链路比较成熟，接口封装也相对直接，因此在系统接入时的适配成本较低，它也可以与 DRENet 在误检控制上形成互补。

4.5 定性样例与误差分析

定量指标只能给出整体趋势，定性样例可以进一步呈现不同错误类型。本文整理成功检测、漏检和误检样例，并结合表 4.2 的指标解释模型行为。

4.5.1 成功检测样例

在成功样例中，模型 DRENet 在海面纹理较弱、目标尺寸较小的图像上仍能给出响应，与其召回偏高的定量结果相符，而 YOLO26 的结果更加整洁，尤其在多目标和常规场景中，预测框与标注框的匹配结果更稳定，同时 FCOS 也能在部分样例中完成定位，但其波动更大。如图 4.4 所示，各模型在本文代表样本上的检测风格差异较大。

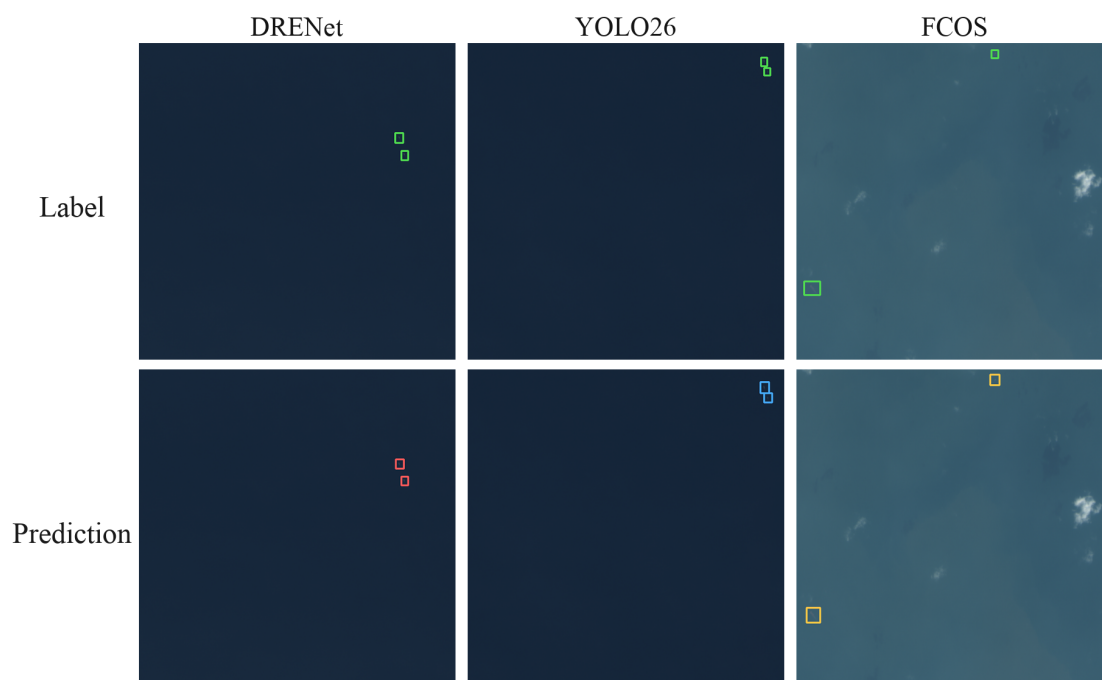


图 4.4 三模型成功检测样例对比

4.5.2 漏检样例

在漏检样例中,三类模型都会遗漏目标,但结果的遗漏位置和原因不同,如图 4.5 所示,结果的标注框与预测框之间的差异,主要是出现在弱纹理、小尺寸以及背景对比度不足的区域。

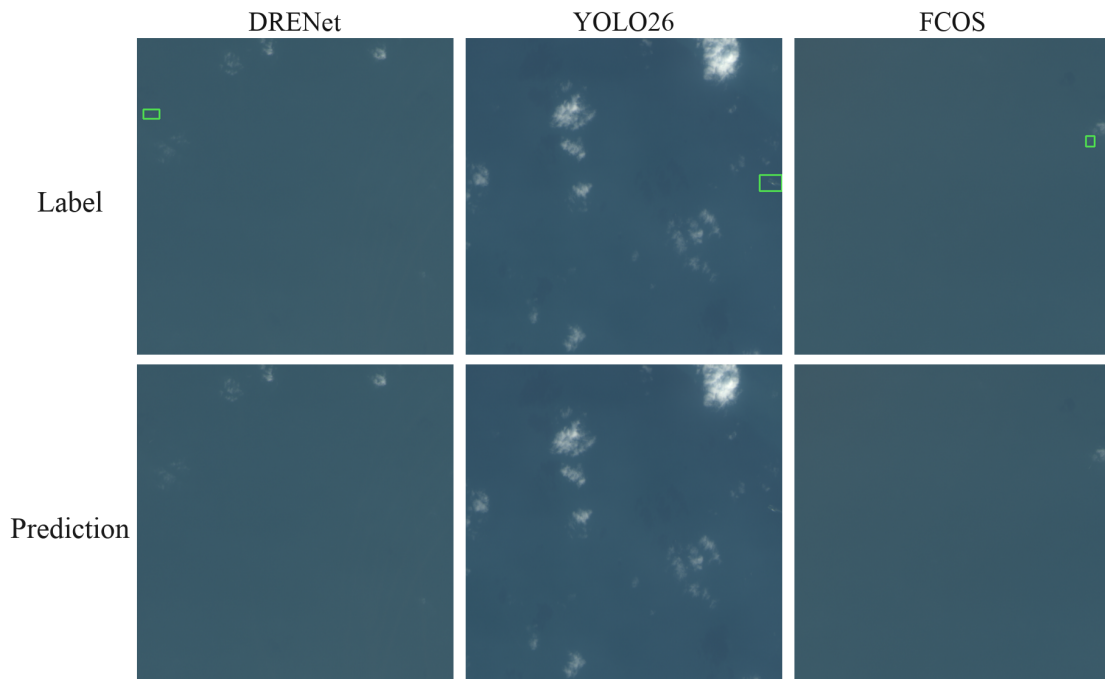


图 4.5 三模型漏检样例对比

4.5.3 误检样例

误检样例则更能解释 Precision 结果的差异，DRENet 更加容易把复杂背景纹理保留下来，而 YOLO26 偶尔会在边缘背景区域产生额外检测结果；同时 FCOS 也存在误检，但数量和位置与前两者不同，如图 4.6 所示，三者误检分布位置并不一致。

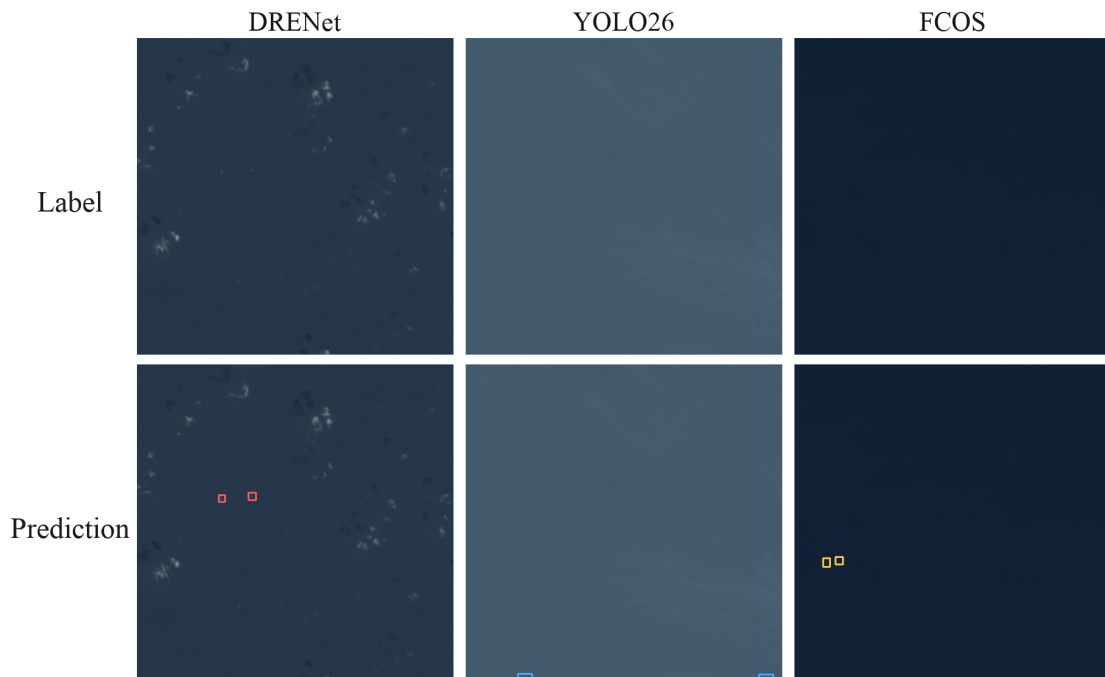


图 4.6 三模型误检样例对比

把这些样例和表 4.2 对照后可以发现,三模型的差别并不是单纯分数高低。DRENet 更偏向“先检出”,复杂背景下误检会变多;YOLO26 更偏向输出稳定和边框质量,弱目标场景下可能漏掉一部分目标;FCOS 的 Precision-Recall 折中较好,但检出能力和效率均弱于前两者。

4.6 消融实验

实验部分,为了说明主表中 Precision 与 Recall 的差异,主要是补充置信度阈值和输入尺寸两组消融,前者是为了观察候选框筛选变化后各模型的响应,后者观察推理输入尺寸变化对检测结果的影响。

4.6.1 阈值消融分析

阈值消融固定 IoU 阈值为 0.50, 置信度阈值取 0.15、0.25 和 0.35。本文主要观察阈值升高后 Precision、Recall 和 F1 的变化情况。三模型结果见表 4.4。

表 4.4 三模型阈值消融结果

模型	设置	AP50	Precision	Recall	F1	结论
DRENet	conf=0.15	0.6616	0.6441	0.7935	0.7110	低阈值召回更高，但误检增加
DRENet	conf=0.25	0.6616	0.7331	0.7663	0.7493	默认值更均衡
DRENet	conf=0.35	0.6616	0.7778	0.7355	0.7561	精度继续上升，召回下降
FCOS	conf=0.15	0.7389	0.7283	0.8351	0.7781	低阈值检出更充分
FCOS	conf=0.25	0.7389	0.7621	0.8297	0.7944	默认值已较均衡
FCOS	conf=0.35	0.7389	0.7761	0.8225	0.7986	F1 略优于 0.25
YOLO26	conf=0.15	0.6661	0.6412	0.7899	0.7078	召回提升明显，但误检偏多
YOLO26	conf=0.25	0.6661	0.7527	0.7609	0.7568	综合表现最均衡
YOLO26	conf=0.35	0.5914	0.8248	0.6993	0.7569	精度最高，但 AP50 与召回下降

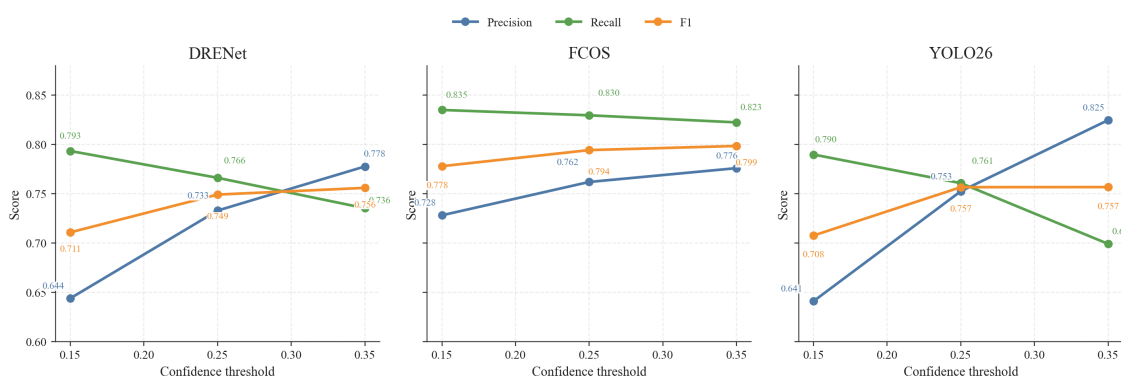


图 4.7 三模型阈值消融趋势图

表 4.4 和图 4.7 显示，阈值升高后，三模型的 Precision 大多上升，Recall 大多下降。这一趋势符合候选框筛选逻辑，也说明后处理阈值会明显改变遥感微小船舶检测结果。

DRENet 对阈值的变化较为敏感，低阈值会进一步放大它的召回倾向，但误检也随之增加；而当阈值提高到 0.35，Precision 升到了 0.7778，Recall 降至 0.7355，若系统需要在检出率和误检数量之间取一个中间位置，0.25 相比 0.15 和 0.35 更适合作为默认值。

FCOS 和 YOLO26 的变化则更加细致，FCOS 在当前稳定权重下，阈值由 0.25 提高到 0.35 时，F1 从 0.7944 小幅度升至 0.7986，说明这组权重下略高阈值能减少

一部分冗余候选，YOLO26 在 0.25 时 AP50、Precision 和 Recall 的组合更均衡；继续升高到 0.35 后，Precision 为 0.8248，但 AP50 与 Recall 同时下降，所以综合 AP50、Precision、Recall 和 F1，0.25 更加适合作为后续实验和系统推理的默认阈值。

4.6.2 输入尺寸敏感性分析

输入尺寸敏感性分析固定置信度阈值为 0.25、IoU 阈值为 0.50，只改变推理阶段的输入尺寸。DRENet 的推理流程对输入尺寸有固定约束（模型内部将输入缩放至预设分辨率后输出固定尺寸特征图，不支持自定义推理尺寸），因此本节重点比较 FCOS 与 YOLO26 在不同输入下的响应差异，结果见表 4.5。

表 4.5 FCOS 与 YOLO26 输入尺寸敏感性分析结果

模型	设置	AP50	Precision	Recall	F1	结论
FCOS	imgsz=512	0.4490	0.5903	0.6395	0.6139	输入过小时性能受限
FCOS	imgsz=640	0.6452	0.7405	0.7808	0.7601	相比 512 提升明显
FCOS	imgsz=800	0.7389	0.7621	0.8297	0.7944	本组综合表现最优
YOLO26	imgsz=512	0.6661	0.7527	0.7609	0.7568	基线稳定
YOLO26	imgsz=640	0.6737	0.7543	0.7953	0.7743	综合表现最优
YOLO26	imgsz=800	0.6564	0.6827	0.7989	0.7362	召回提高但误检增加

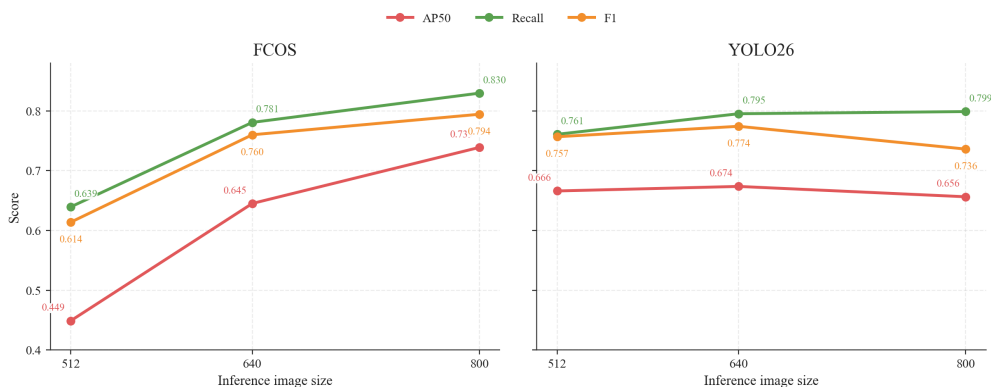


图 4.8 输入尺寸敏感性趋势图

上述数据显示，FCOS 对输入尺寸更敏感，512 到 800 的 AP50、Precision、Recall 和 F1 均呈上升趋势；YOLO26 在 640 输入下取得本组最高 AP50 和 F1，Precision 与 Recall 也保持较好平衡。当输入继续增至 800 时，YOLO26 的 Recall 略升，但 Precision 和 F1 明显下降，说明更大输入会带来更多候选目标，同时也更容易引入误检。

5 系统需求分析、设计与实现

5.1 系统建设目标

系统部分围绕遥感微小船舶检测任务来展开，目标是实现图像上传、模型推理、任务管理、结果可视化和历史记录查询等功能。系统整体需要覆盖前端交互、后端服务、数据持久化和算法接入几个环节，使本实验结果能够以任务形式进行管理和复查。

最终本文整体系统使用 React + FastAPI + SQLite 的前后端分离架构，前端使用 React 负责任务提交、状态查询和结果展示；后端 FastAPI 组织 HTTP 接口并调用模型推理代码；使用 SQLite 数据库保存任务、结果和产物路径，该架构结构清晰，同时也便于后续接口维护和结果追踪。

5.2 系统需求分析

5.2.1 功能需求

结合课题任务，系统需要覆盖以下功能。

1. **模型管理**：可读取模型配置和展示模型名称、状态和权重信息，并且支持模型启用或停用；
2. **任务提交**：能够支持单张图像与批量图像上传，并允许用户设置模型类型和置信度阈值；
3. **任务执行**：用异步的方式执行推理任务，用于避免前端界面阻塞，且支持任务状态查询；
4. **结果展示**：展示检测框可视化图像、结构化检测结果表格及统计信息；
5. **结果持久化**：在系统重启后，仍可保留历史任务的结果和输出文件路径，以支持任务复查。

表 5.1 为各操作场景的响应定义，用于约束前后端联调时的行为一致性。

表 5.1 功能需求的操作—响应定义

操作场景	用户操作	期望系统响应
模型管理	启用/停用模型	页面状态即时更新，后端返回成功状态并持久化
单图任务提交	上传 1 张图并提交	返回 <code>task_id</code> ，任务状态进入 <code>queued/running</code> ，跳转详情页
批量任务提交	上传多图并提交	返回 <code>task_id</code> ，列表与详情页显示总数与完成数进度
任务执行失败	模型路径错误或输入异常	状态变为 <code>failed</code> ，并在详情页返回可读错误信息
结果回看	打开历史任务详情	展示任务参数、可视化图、结构化结果和统计图
结果导出	点击 CSV 导出	生成并下载包含 <code>bbox/score/-source_model</code> 的结果文件

5.2.2 非功能需求

非功能需求主要包括可维护性、可复现性和稳定性。环境安装、数据库初始化和项目启动方式需要可重复；算法层、服务层和展示层的边界要清楚，后续替换模型时不至于牵动整套页面；界面信息需要能够清楚呈现任务状态、模型参数和检测结果。

5.3 系统总体架构

系统采用前后端的组织分离架构，如图 5.1 所示，前端页面用于处理任务创建、列表查询和结果可视化，后端为前端提供接口服务、任务调度、状态管理和历史文件访问；而数据库则用于保存模型、任务、检测结果和输出文件路径；算法能力层用于封装整体模型预测服务、模型适配器与融合逻辑，并由后端通过统一接口调用。

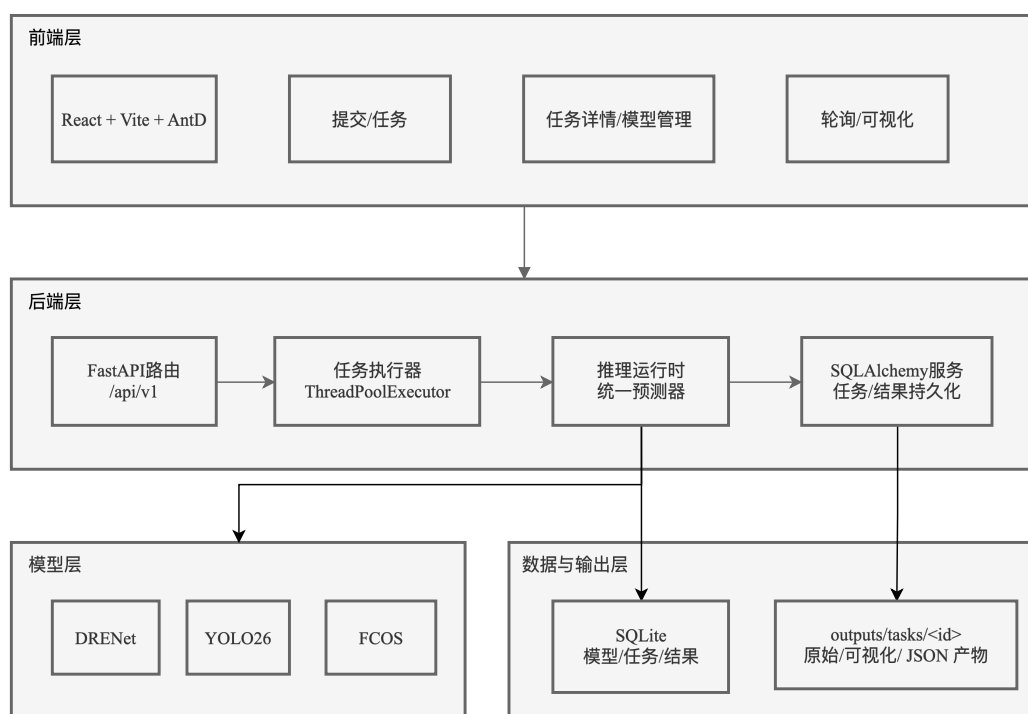


图 5.1 系统总体架构图

从职责划分角度看，系统可分为以下五层：

1. **前端展示层**：使用 React、Vite、TypeScript 与 Ant Design 技术栈，实现任务提交页、任务列表页、任务详情页和模型管理页；
2. **后端服务层**：使用 FastAPI 提供任务提交、任务查询、结果查询、模型查询和健康检查等接口；
3. **任务调度层**：使用 ThreadPoolExecutor 实现异步任务执行，以降低界面阻塞；
4. **数据持久化层**：使用 SQLite 与 SQLAlchemy 技术栈来保存模型信息、任务状态、检测结果与文件路径；
5. **算法能力层**：用于封装预测函数服务、模型适配器与融合逻辑以实现单模型与融合模式推理。

图 5.1 中的分层体现了系统各模块的职责边界，前端只和 HTTP 接口交互，不直接接触模型文件；而后端通过统一推理入口调用 DRENet、YOLO 或融合逻辑；数据库和输出目录则共同保存状态与产物路径，后续替换模型时，可以优先修改算法能力层和配置文件。

5.4 数据库设计

根据任务追踪和结果回看功能，数据库的核心表包括 `models`、`tasks`、`results` 和 `task_files`，`models` 表保存模型名称、键值、权重路径和启停状态；`tasks` 表记录任务类型、执行模式、状态和时间；`results` 表保存检测框结构化结果；`task_files` 表记录输入图、输出 JSON 和可视化结果路径。

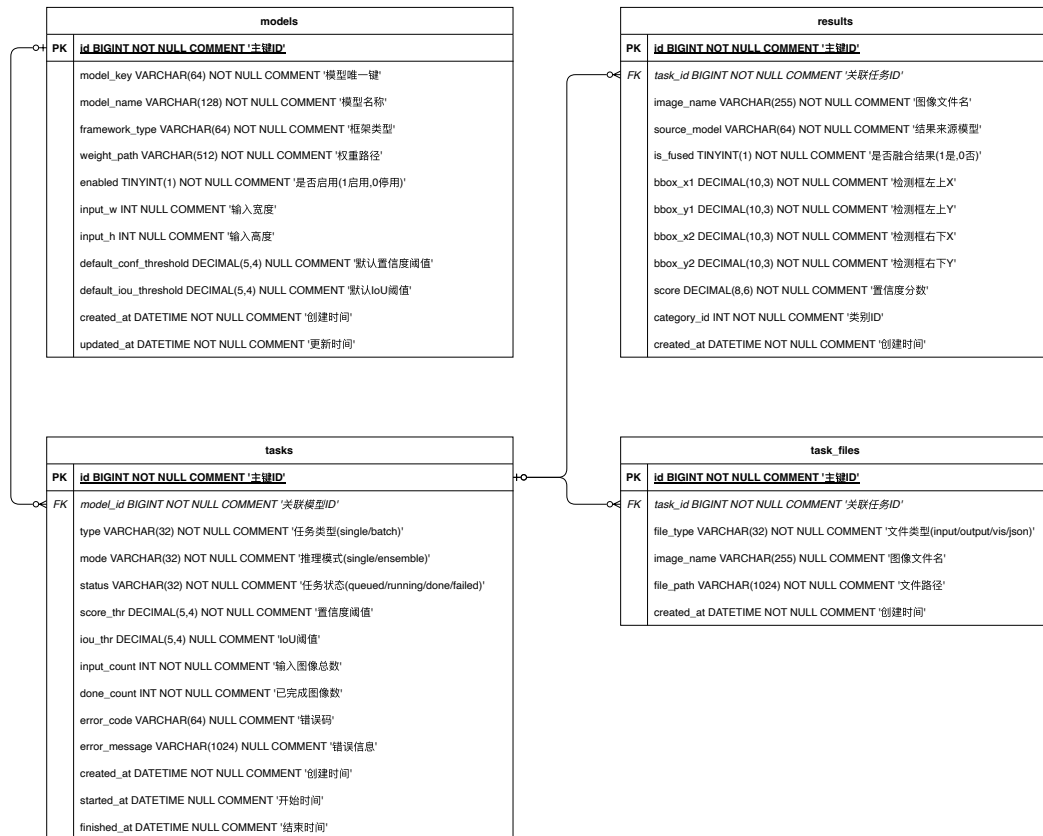


图 5.2 系统数据库 E-R 图

图 5.2 展示了四张表之间的关系，`tasks` 表位于任务链路中心，一条任务可以对应多条检测框结果，也可以对应多个输入或输出文件记录，`models` 表同时服务前端模型管理页和后端推理校验，通过组合这些表，系统可以记录从任务创建到结果回看的完整链路。

5.5 接口设计

后端接口挂载在 `/api/v1` 路径下，主要接口见表 5.2。

后端接口主要覆盖任务提交、任务查询、结果查询、模型查询和健康检查，且

表 5.2 系统核心接口设计

接口路径	请求方式	功能说明
/api/v1/tasks/infer	POST	提交单图或批量推理任务，返回任务编号与初始状态
/api/v1/tasks	GET	查询任务列表、状态、进度与时间信息
/api/v1/tasks/<task_id>	GET	查询单个任务的参数、状态与错误信息
/api/v1/tasks/<task_id>/progress	GET	查询任务进度，返回状态与完成计数，用于高频轮询
/api/v1/tasks/<task_id>/results	GET	查询任务结果、可视化文件路径和结构化检测框
/api/v1/models	GET	查询模型列表与启停状态
/api/v1/models/<model_key>	PATCH	更新模型启停状态
/api/v1/health	GET	查询服务健康状态与模型加载情况

路径统一管理在 /api/v1 下，可供前端调用集中，也方便后续接口维护。

5.6 关键实现流程

5.6.1 任务创建与执行流程

任务从用户在前端上传图像开始。选择模式、模型和阈值之后，前端调 /api/v1/tasks/infer。后端收到请求先校验图像、阈值和模型配置，开始建立任务记录写进数据库。然后任务交给线程池执行器，执行器根据参数选单模型或者融合模式，再调统一推理接口跑检测。

任务状态记录为 `queued`、`running`、`done` 和 `failed` 四种，前端列表页和详情页都读这些状态来展示任务的生命周期；推理或文件处理如果出错，后端会把错误信息写回任务记录，方便定位失败原因。

5.6.2 结果生成与展示流程

推理完成后，结构化检测结果写入数据库，输入图、输出 JSON 和可视化图像保存到 `outputs/tasks/<task_id>/`。任务运行期间，前端优先轮询 /api/v1/tasks/<task_id>/progress 接口，只获取 `status`、`done_count` 和 `input_count` 等字段；

任务结束后，再请求结果接口加载完整检测框和可视化产物。任务执行与结果展示流程如图 5.3 所示。

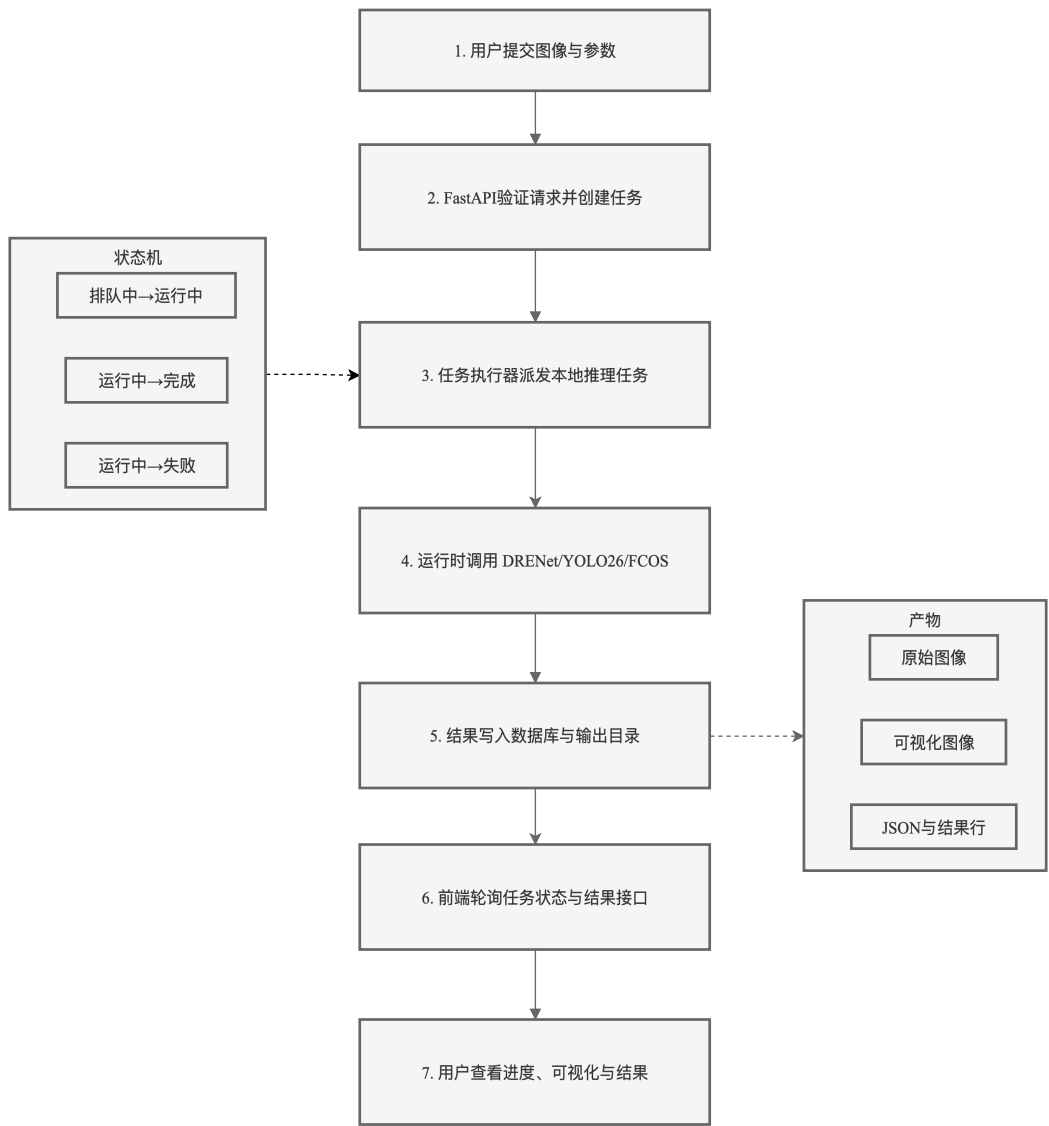


图 5.3 系统关键流程图

5.7 模块实现细节

5.7.1 端到端数据流与职责分解

本文将一次任务的端到端链路拆分为四段：前端请求构建、后端推理编排、结果持久化和可视化回传，前端提交参数时，会统一封装模型键值、阈值、执行模式和输入文件；后端接口层负责参数校验与任务建档；调度层在后台线程中调用预测

服务并聚合检测框；存储层将任务状态、检测结果和产物路径落盘，最后由结果接口把结构化结果和可视化文件返回给前端页面完成展示，该分段设计的目的是：接口层、算法层、存储层可以独立维护，后续替换模型时不必改动前端交互协议。

5.7.2 单图检测逻辑（接口层）

接口层的核心职责是“参数标准化 + 任务创建 + 异步投递”，其核心逻辑如下代码所示。

```
def submit_infer_task(request):  
    payload = normalize_request(request)  
    validate_model_and_threshold(payload)  
    task = task_repo.create(  
        mode=payload.mode,  
        model_key=payload.model_key,  
        conf=payload.conf,  
        iou=payload.iou,  
        status="queued"  
    )  
    executor.submit(run_infer_pipeline, task.id, payload)  
    return {"task_id": task.id, "status": "queued"}
```

该逻辑保证了前端拿到任务编号后即可进入轮询，不需要阻塞等待推理完成，即使后续任务失败，也能通过 `task_id` 在详情页定位错误信息，提升了系统可观测性。

5.7.3 推理调用逻辑（服务层）

服务层聚焦“统一预测入口 + 多模型策略分发”，单模型模式下直接调用指定模型适配器；融合模式下按配置选择串行或并行执行，再将结果交给融合后处理。其核心流程如下代码所示。

```
def run_infer_pipeline(task_id, payload):  
    task_repo.update_status(task_id, "running")
```

```

image_list = prepare_inputs(payload.files)
for image_path in image_list:
    if payload.mode == "single":
        records = predict_service.predict(
            image_path=image_path,
            model_name=payload.model_key,
            conf_threshold=payload.conf,
            iou_threshold=payload.iou
        )
    else:
        records = predict_service.predict_fusion(
            image_path,
            payload
        )

    result_repo.save_records(task_id, image_path, records)
task_repo.update_status(task_id, "done")

```

该设计将“接口协议”与“算法实现”分离：接口层不感知具体模型框架差异，服务层通过适配器完成 DRENet、FCOS、YOLO26 的统一调用，便于后续新增模型。

5.7.4 结果聚合逻辑（存储层）

存储层负责“状态写回 + 文件索引 + 结构化检测框存储”，其目标是确保任务可追溯与结果可复查，对应逻辑如下代码所示。

```

def save_records(task_id, image_path, records):
    vis_path = render_and_save(image_path, records, task_id)
    json_path = dump_json(records, task_id, image_path)
    db.insert_task_file(task_id, "input", image_path)
    db.insert_task_file(task_id, "vis", vis_path)
    db.insert_task_file(task_id, "json", json_path)
    for rec in records:
        db.insert_result(

```

```
task_id, image_path,  
rec["bbox"], rec["score"], rec["label"]  
)
```

通过上述写入流程，系统既能展示可视化图片，也能导出结构化表格，满足“页面复查 + 数据再分析”双重需求。

5.8 系统实现效果

经过前后端联调和真实模型接入测试，系统已经跑通任务创建、推理执行、结果保存和历史回放整体链路。当前系统可以接入 DRENet、YOLO 与 FCOS 模型权重，完成单图和批量推理；任务列表模块支持轮询刷新、进度展示和失败状态标记；任务详情页展示模型来源、检测框、平均置信度和统计图表；结构化检测结果可以导出为 CSV 表格。SQLite 数据库则保存历史任务和检测结果，保证系统重启后仍可查询。

5.8.1 前端页面实现效果

任务提交页支持单图和批量输入，推理模式、模型选择和阈值参数放在同一表单区域，在任务列表页突出状态、进度、模式、创建时间和耗时，方便观察任务执行情况，在任务详情页展示可视化结果、检测框表格、模型来源和统计图表，用于回看单次任务的输入、执行和输出。

相比只展示一张检测结果图，Web 前端可以同时放置任务参数、执行状态、结果图像和结构化检测框。这样，实验结论不只停留在表格和文字中，也能通过系统任务记录进行复查。如图 5.4 所示，任务提交页、任务列表页和任务详情页共同构成了完整的使用链路。

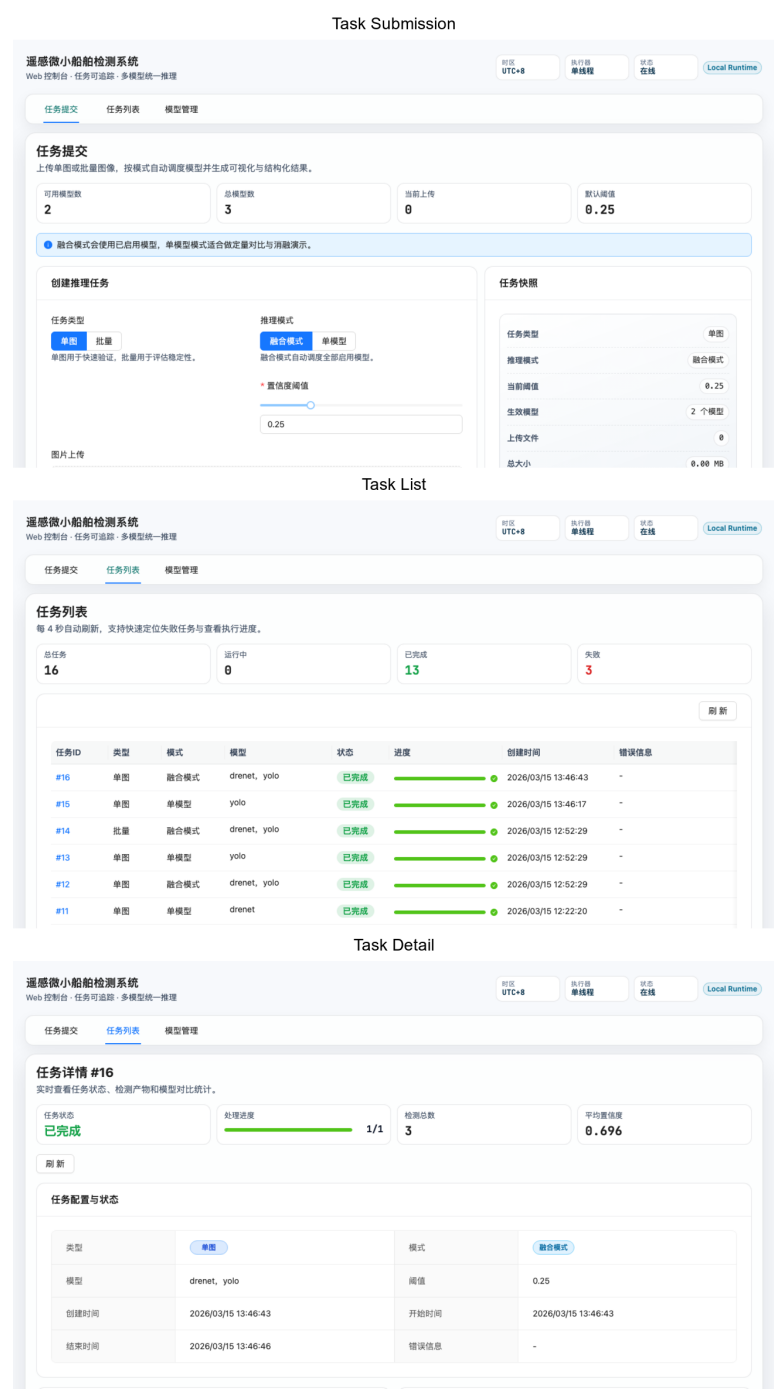


图 5.4 系统主要页面实现效果

5.9 系统输出案例集

本节选取数据集中三组不同样本的系统输出结果图，用于展示系统接入模型后在不同场景下的检测表现，三组样本分别覆盖相对清晰场景、弱纹理场景与复杂背景场景，以便观察模型在目标可见性与背景干扰变化条件下的输出差异。

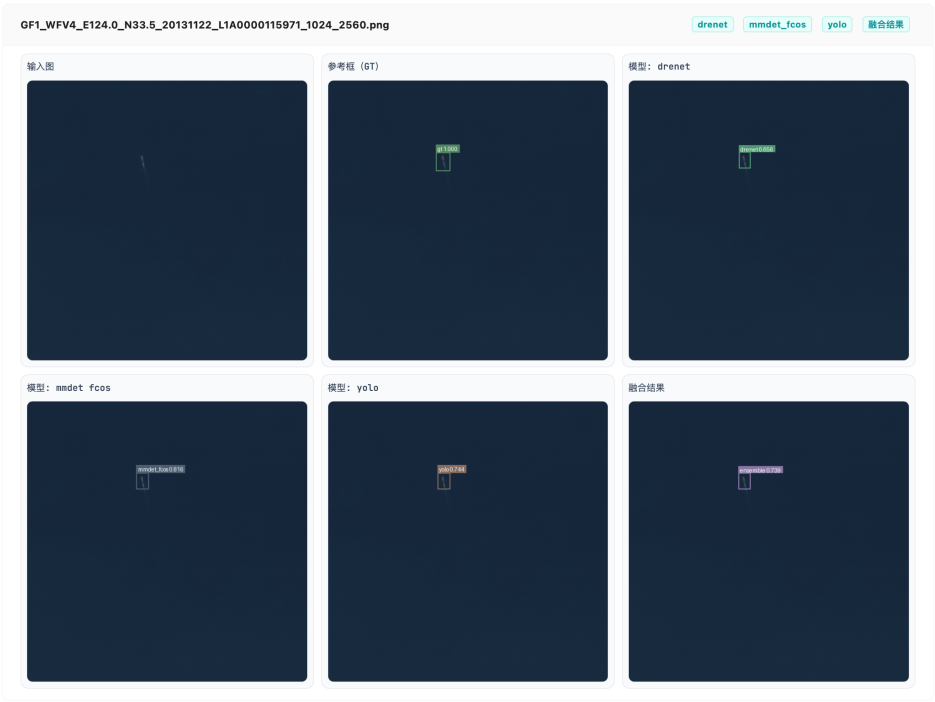


图 5.5 样本 A 的系统输出结果图：目标边界相对清晰场景下的检测结果示例

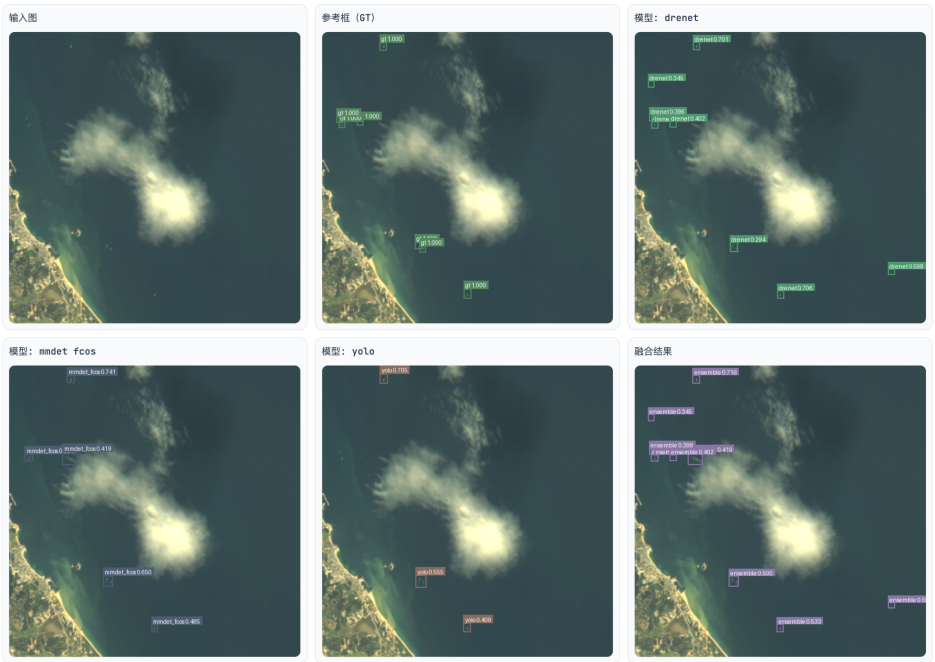


图 5.6 样本 B 的系统输出结果图：弱纹理与低对比度条件下的检测结果示例



图 5.7 样本 C 的系统输出结果图：复杂背景干扰条件下的检测结果示例

结合图 5.5 至图 5.7 可以观察到：当目标边界清晰时，系统输出结果较稳定；而当目标纹理弱、对比度降低时，漏检风险上升；当背景纹理复杂时，误检风险增加，该结果与第四章中的定量与定性分析结论一致，说明系统侧结果展示能够较好反映模型在不同样本条件下的行为差异。

5.10 模块测试用例设计

为验证系统在正常与极端条件下的可用性，本文的测试覆盖了接口层、任务调度层、结果展示层和持久化层，按模块设计的测试用例如表 5.3 所示。

5.10.1 测试执行结果

基于表 5.3 的用例集合，本文完成了功能与异常场景的联调测试，总体结果表明：正常流程下任务提交、异步执行、结果回看和 CSV 导出均可稳定完成；异常流程下空输入、阈值越界、无效任务编号和模型异常均能够返回可解释错误信息，且不会影响其他任务状态。

进一步地，在服务重启后的历史任务复查测试中，任务记录与结果文件索引能够正确恢复；在高频轮询与并发查询场景下，任务状态字段保持一致，未观察到明

表 5.3 系统主要模块测试用例

模块	用例类型	输入/场景	期望结果
任务提交	正常	合法图片 + 合法阈值	返回 <code>task_id</code> ，任务状态进入 <code>queued/running</code>
任务提交	极端	空文件、超限文件、阈值越界	请求被拒绝，返回明确错误信息
任务执行	正常	单图/批量任务	状态按 <code>queued->running->done</code> 演进，结果落盘
任务执行	极端	模型权重缺失、推理中断	状态进入 <code>failed</code> ，错误信息可追溯
结果查询	正常	已完成任务详情查询	返回可视化链接、结构化 <code>bbox</code> 、统计字段
结果查询	极端	不存在 <code>task_id</code>	返回 <code>404</code> 或业务错误码，不影响其他任务
数据持久化	正常	服务重启后查询历史任务	历史任务记录与结果仍可访问
数据持久化	极端	高频轮询与并发查询	状态字段一致，无明显数据错乱

显的数据错乱，上述结果说明当前系统实现能够满足本课题对可用性、可观测性与可追溯性的基本要求。

5.11 系统响应速度优化

系统联调时出现过一个较明显的现象：模型本身推理耗时并不高，但端到端页面返回结果的时间相对更长。排查后发现，等待时间并不主要花费在检测模型上，而是分散在融合模式串行调用、重复图像解码、运行期间拉取完整结果、数据库频繁提交和结果接口重复解析等数据传输环节。因此，本文将优化重点放在端到端响应时间上，而不是只看单个模型函数的耗时。

5.11.1 性能瓶颈分析与优化思路

在对当前系统的任务提交、推理执行和结果展示三段流程检查后，发现主要性能开销集中在七处：融合模式下多模型默认串行；任务运行期间前端反复请求完整结果；可视化阶段对同一张大图多次执行 `Image.open`；批量任务逐图提交数据库事务；结果接口重复解析数据集信息和参考框；任务输入阶段复制源图；首次推理时

才加载模型权重。

对应改动也按这几处展开：融合推理增加了可配置并发执行；前端任务页面轮询改为 `progress` 接口；可视化阶段直接传递 `PIL.Image` 对象；批量任务改为分段提交；数据集信息和参考框加入内存缓存；输入文件优先使用符号链接；应用启动时预加载启用模型。这些改动的共同目标是减少端到端整体流程中的等待时间。

5.11.2 并行推理与模型预加载机制

融合模式需要对于同一张图像调用多个模型，再合并其检测结果。初始实现按顺序执行，总耗时接近各模型耗时之和。为减少这部分的等待时间，后端推理运行时加入了基于 `ThreadPoolExecutor` 的并行推理机制，并通过环境配置控制并行模型数。并行度设为 1 时仍按串行执行；而当 GPU 资源较充足时，可以选择性地提高并行度来缩短融合推理时间。融合后处理阶段采用加权框融合思路统一多模型候选框^[24]，并与常见 NMS 系列方法的设计取向保持一致^[25]。

并行并没有被设为默认开启。原因是线程争抢会使单模型耗时上升，三模型并行也可能带来显存竞争。本文因此保留默认并行度为 1，只在资源允许时手动提高并行度，让系统在不同机器上更容易稳定运行。

另外，优化后的系统在 `startup` 阶段预加载已启用模型权重，避免第一次提交任务时才初始化模型。这个改动并不改变稳态的吞吐量，但是能够减少首次推理的额外等待时间。

5.11.3 进度端点与前端轮询重构

优化前，任务详情页每 4 秒轮询一次，并同时请求任务详情和完整结果，该实现简单但体验不够理想：任务实际完成后，页面最多还要等待 4 秒才会刷新，且任务运行期间反复请求结果接口也会产生不必要的数据传输和后端解析开销。

针对该问题，后端新增进度查询接口，只返回 `status`、`done_count` 和 `input_count` 等字段，而前端改为两阶段轮询：当任务处于 `running` 时，每 1.5 秒请求 `progress` 接口，只更新任务进度条和状态；当任务转为 `done` 或 `failed` 后，前端再请求结果接口，并加载完整检测数据和可视化产物，高频轮询响应由约 25KB 缩减到约 64B，页面感知延迟也随之降低。

5.11.4 渲染、存储与结果访问优化

可视化渲染阶段原本会多次打开同一张图，本文扩展了 `render_detections()` 的输入方式，让它既能接收图像路径，也能直接接收 `PIL.Image` 对象，所以在融合模式下，系统只需打开一次原始图像，就能完成多个模型结果和融合结果的渲染。

针对批量任务，数据库的提交策略也做了调整，原先每处理一张图就立即提交，现在改为每处理两张图或到达任务末尾时提交一次，这样改动既保留阶段性进度，又减少事务提交次数，同时在结果接口另外加入数据集信息和参考框缓存，避免同一张图片多次查询时重复遍历目录和解析标注。

在输入文件准备阶段优先使用符号链接代替物理复制，用以减少大图 I/O；如果符号链接创建失败，可再回退到复制方案，上述改动能一起缩短任务从提交到结果展示之间的实际等待。

5.11.5 优化效果分析

本文为检查这些改动是否有效，对于几个关键环节做了微基准测试，结果见表 5.4。

表 5.4 系统响应速度优化的关键基准结果

优化项	优化前	优化后	提升倍数	说明
三模型融合推理	465.6 ms	156.2 ms	2.98×	3 workers 并行相对串行
前端结果可感知延迟	4.0 s	1.5 s	2.67×	运行期间改为 progress 轮询
大图可视化渲染	443.9 ms	336.1 ms	1.32×	4096×4096 图像由 4 次解码降为 1 次
批量任务数据库提交	62.6 ms	37.6 ms	1.66×	10 张图任务由逐图提交改为分段提交
输入文件准备	2.4 ms	0.8 ms	3.00×	优先 symlink，失败时回退 copy

表 5.4 里的收益大致来自两类改动：一类是减少串行等待，比如融合推理并行和 progress 轮询；另一类是减少重复工作，比如图像传递优化、结果缓存和分段提交。这些数据 and 实际联调时页面响应变快的方向是一致的，说明优化思路是有效的。

6 总结与展望

6.1 工作总结

本文围绕遥感图像中的微小船舶检测，完成了文献梳理、模型实验和系统实现几项工作。研究部分先整理通用目标检测、小目标检测和遥感船舶检测的发展历史，再在同一数据和评测口径下比较 DRENet、FCOS 与 YOLO26 模型。工程部分采用 React、FastAPI 和 SQLite 搭建图像检测系统，完成任务提交、异步推理、结果可视化和历史回看等功能。

实验部分的主要结论是，DRENet 和 YOLO26 的 AP50 非常接近，但取向不同。DRENet 的 Recall 更高，适合需要优先发现目标的场景；YOLO26 在 AP50-95、Precision 和 F1 上更加均衡；FCOS 的 Precision-Recall 折中较好，但整体检出能力和效率弱于前两者。阈值消融和输入尺寸敏感性分析也充分说明，误检和漏检不能只靠一个 AP50 指标解释。

系统部分并不是单独做一个结果展示页，而是把推理能力、任务状态、结果文件和历史记录放到同一套流程里管理。经过联调后，系统已经可以接入真实模型权重，支持任务回放，离线实验中的部分结果也能在页面中复查。

本文的工作重点不在提出新的检测网络，而在于把数据口径、实验记录和系统实现整理成一条可复查的路径。

6.2 不足分析

当前工作已经完成主要目标，但仍有以下几处不足。

第一，DRENet 的推理流程不支持自定义输入尺寸，导致输入尺寸消融实验只能覆盖 FCOS 和 YOLO26，三模型在该维度上的比较不够完整。第二，FCOS 的训练配置（如 backbone 选择、数据增强策略）与 DRENet 和 YOLO26 不完全对齐，可能对最终指标产生一定影响。第三，系统目前仅在本地环境完成联调，尚未在多平台或真实部署场景下验证稳定性和跨环境迁移能力。第四，消融实验仅覆盖置信度阈值和输入尺寸两个维度，未涉及数据增强、损失函数和后处理策略等其他可能影响模型行为的因素。

这些问题不影响本文对三模型主要结论的有效性，但若要进一步提高评测严谨性和系统可迁移性，后续应重点补齐 DRENet 的尺寸消融、统一 FCOS 的训练配置、

完善部署流程，并扩展消融维度。

6.3 未来展望

后续研究可从以下方面展开：把融合模式的统一离线评测补齐，把单模型结果和融合展示之间的关系量化出来；继续围绕 DRENet 和 YOLO26 的误检、漏检取舍来调阈值、后处理和融合策略；继续整理 FCOS 的训练配置，让跨模型比较更严谨；完善模型接入和部署流程；在更多遥感场景和数据集上检查方法和系统的泛化情况。

沿着这些方向继续做，现有系统可以进一步扩展成实验管理平台。比如把统一报告导出、批量回评和参数留痕都接到同一个工作流里，让实验复查和系统展示共享更多底层能力。

6.4 全文结论

本文完成了遥感微小船舶检测任务中的实验比较和系统实现。在 LEVIR-Ship 数据集上，DRENet 更偏召回优先，YOLO26 更偏精度和部署均衡，FCOS 的 Precision-Recall 折中较好但检出能力偏弱。系统实现则说明，将推理能力服务化并纳入任务管理后，结果展示和历史回放会更加清楚。

从最终完成情况看，本文已经把问题分析、实验论证和系统实现串联起来。现有实验记录、分析框架和系统代码，为后续性能优化、评测补齐和部署规范化留下了可继续迭代的基础。

参考文献

- [1] Shaoqing Ren, Kaiming He, Ross Girshick, Jian Sun. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks[C]//Advances in Neural Information Processing Systems: vol. 28. 2015.
- [2] Tsung-Yi Lin, Piotr Dollar, Ross Girshick, Kaiming He, Bharath Hariharan, Serge Belongie. Feature Pyramid Networks for Object Detection[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2017: 2117-2125.
- [3] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, Piotr Dollar. Focal Loss for Dense Object Detection[C]//Proceedings of the IEEE International Conference on Computer Vision (ICCV). 2017.
- [4] Zhi Tian, Chunhua Shen, Hao Chen, Tong He. FCOS: Fully Convolutional One-Stage Object Detection[C]//Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). 2019.
- [5] Joseph Redmon, Santosh Divvala, Ross Girshick, Ali Farhadi. You Only Look Once: Unified, Real-Time Object Detection[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2016.
- [6] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, Sergey Zagoruyko. End-to-End Object Detection with Transformers[Z]. 2020.
- [7] Xizhou Zhu, Weijie Su, Lewei Lu, Bin Li, Xiaogang Wang, Jifeng Dai. Deformable DETR: Deformable Transformers for End-to-End Object Detection[Z]. 2021.
- [8] Xuan Wang, Aoran Wang, Jinglei Yi, Yongchao Song, Abdellah Chehri. Small Object Detection Based on Deep Learning for Remote Sensing: A Comprehensive Review[J]. Remote Sensing, 2023, 15(13): 3265.
- [9] Zhe Zhao, Meng Wang, Minglei Zhao, et al. Ship Detection with Deep Learning in Optical Remote-Sensing Images: A Survey of Challenges and Advances[J]. Remote Sensing, 2024, 16(7): 1145.
- [10] Jianqi Chen, Keyan Chen, Hao Chen, Zhengxia Zou, Zhenwei Shi. A Degraded Reconstruction Enhancement-Based Method for Tiny Ship Detection in Remote Sensing Images With a New Large-Scale Dataset[J]. IEEE Transactions on Geoscience and Remote Sensing, 2022, 60: 1-14.
- [11] Ross Girshick, Jeff Donahue, Trevor Darrell, Jitendra Malik. Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2014: 580-587.
- [12] Joseph Redmon, Ali Farhadi. YOLOv3: An Incremental Improvement[Z]. 2018.
- [13] Mingxing Tan, Ruoming Pang, Quoc V. Le. EfficientDet: Scalable and Efficient Object Detection[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2020: 10781-10790.
- [14] Hao Zhang, Feng Li, Shilong Liu, Lei Zhang, Hang Su, Jun Zhu, Lionel M. Ni, Heung-Yeung Shum. DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection [Z]. 2023.

- [15] Xue Yang, Jirui Yang, Junchi Yan, Yue Zhang, Tengfei Zhang, Zhi Guo, Xian Sun, Kun Fu. SCRDet: Towards More Robust Detection for Small, Cluttered and Rotated Objects[C]// Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). 2019.
- [16] Bharat Singh, Larry S. Davis. An Analysis of Scale Invariance in Object Detection – SNIP[C]// Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2018: 3578-3587.
- [17] Bharat Singh, Mahyar Najibi, Larry S. Davis. SNIPER: Efficient Multi-Scale Training[C]// Advances in Neural Information Processing Systems: vol. 31. 2018.
- [18] Mahyar Najibi, Bharat Singh, Larry S. Davis. AutoFocus: Efficient Multi-Scale Inference[C]// Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). 2019.
- [19] Gong Cheng, Junwei Han. A Survey on Object Detection in Optical Remote Sensing Images[J]. ISPRS Journal of Photogrammetry and Remote Sensing, 2016, 117: 11-28.
- [20] Kai Chen, Jiaqi Wang, Jiangmiao Pang, Yuhang Cao, Yu Xiong, Xiaoxiao Li, Shuyang Sun, Wansen Feng, Ziwei Liu, Jiarui Xu, Zheng Zhang, Dazhi Cheng, Chenchen Zhu, Tianheng Cheng, Qijie Zhao, Buyu Li, Xin Lu, Rui Zhu, Yue Wu, Jifeng Dai, Jingdong Wang, Jianping Shi, Wanli Ouyang, Chen Change Loy, Dahua Lin. MMDetection: Open MMLab Detection Toolbox and Benchmark[Z]. 2019.
- [21] Ultralytics. Ultralytics YOLO Docs[EB/OL]. 2024 [2026-03-15]. <https://docs.ultralytics.com/>.
- [22] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollar, C. Lawrence Zitnick. Microsoft COCO: Common Objects in Context[C]//Proceedings of the European Conference on Computer Vision (ECCV). 2014: 740-755.
- [23] Mark Everingham, Luc Van Gool, Christopher K. I. Williams, John Winn, Andrew Zisserman. The PASCAL Visual Object Classes (VOC) Challenge[C]//International Journal of Computer Vision: vol. 88: 2. 2010: 303-338.
- [24] Roman Solovyev, Weimin Wang, Tatiana Gabruseva. Weighted Boxes Fusion: Ensembling Boxes from Different Object Detection Models[J]. Image and Vision Computing, 2021, 107: 104117.
- [25] Navaneeth Bodla, Bharat Singh, Rama Chellappa, Larry S. Davis. Soft-NMS – Improving Object Detection with One Line of Code[C]//Proceedings of the IEEE International Conference on Computer Vision (ICCV). 2017: 5561-5569.

致谢

这篇论文从选题到定稿，前后经历了实验、联调和反复修改，过程中得到过不少帮助。写到这里，还是想把最直接的感谢留给一直支持我的人。

最想感谢的是指导教师。无论是选题方向、实验安排，还是系统实现和论文修改，老师都给了很具体的意见。很多原本比较散的思路，最后能收拢成现在这个版本，离不开这些提醒。

同样也很感谢在实验和系统联调整个过程中帮过我的同学和朋友。模型训练、环境配置、页面体验和结果分析里碰到过不少小问题，都是和他们一起发现和解决的。

由于时间和能力有限，论文中仍然可能有不少不足，欢迎老师批评指正。

作者：邝黄硕

2026 年 5 月 30 日